



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

КУРСОВАЯ РАБОТА
по дисциплине «Разработка клиент-серверных мобильных приложений»

Тема курсовой работы: «Разработка клиент-серверного мобильного приложения социальной сети местоположений»

Студент группы ИКБО-64-23 Алимханов Дени Русланович


(подпись)

Руководитель
курсовой работы

Старший преподаватель
Дворникова Е.М.


(подпись)

Работа представлена к защите «08» 06 2026 г.

Допущен к защите «08» 06 2026 г.

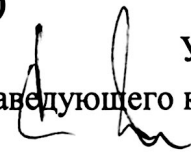
Исабеков, /Исабеков, А.А./

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

Утверждаю
И.о. заведующего кафедрой МОСИТ

Головин С.А.
«16» февраля 2026 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Разработка клиент-серверных мобильных приложений»

Студент Алимханов Дени Русланович

Группа ИКБО-64-23

Тема «Разработка клиент-серверного мобильного приложения социальной сети местоположений»

Исходные данные: Разрабатываемый прототип мобильного приложения должен предоставлять возможности полностью интерактивной системы с понятным дружественным UI и обеспечивать необходимую функциональность, которая формируется в зависимости от заданной темы и предметной области изучаемых вопросов.

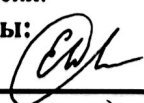
Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Описание предметной области мобильного приложения;
2. Сравнительный анализ существующих аналогов мобильного приложения;
3. Обоснование выбора инструментальных средств разработки мобильного приложения;
4. Определение функциональных и нефункциональных требований;
5. Проектирование пользовательского интерфейса мобильного приложения;
6. Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов);
7. Проектирование базы данных мобильного приложения;
8. Разработка клиентской части мобильного приложения;
9. Разработка серверной части мобильного приложения;
10. Тестирование разработанного мобильного приложения.

Отчет по курсовой работе в виде пояснительной записки.

Срок представления к защите курсовой работы:

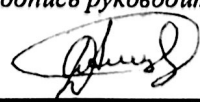
Задание на курсовую работу выдал


Подпись руководителя

до «30» мая 2026 г.

Дворникова Е.М.
(ФИО руководителя)

Задание на курсовую работу получил


Подпись обучающегося

«16» февраля 2026 г.
Алимханов Д.Р.
(ФИО обучающегося)
«16» февраля 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ	6
1.1 Описание предметной области мобильного приложения	6
1.2 Сравнительный анализ существующих аналогов	8
1.3 Обоснование выбора инструментальных средств для разработки мобильного приложения	10
2 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ	14
2.1 Определение функциональных и нефункциональных требований	14
2.2 Проектирование пользовательского интерфейса	16
2.3 Логическая модель приложения	20
2.3.1 Проектирование мобильного приложения	20
2.3.2 Проектирование базы данных	24
3 РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ	27
3.1 Разработка клиентской части мобильного приложения	27
3.2 Разработка серверной части мобильного приложения	30
3.2.1 Описание основной логики сервера	30
3.2.2 Подключение базы данных к серверу	33
3.2.3 Подключение сервера к клиентской части мобильного приложения	34
3.3 Тестирование разработанного приложения	36
ЗАКЛЮЧЕНИЕ	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	41
ПРИЛОЖЕНИЯ	44
Приложение А. Листинг серверной части приложения	45
Приложение Б. Листинг клиентской части приложения	63
Приложение В. Экраны мобильного приложения	100
Приложение Г. Тестовые сценарии	105

ВВЕДЕНИЕ

Мобильные устройства стали основным способом доступа к информации о городской среде: пользователи ищут кафе, парки, смотровые площадки и спортивные объекты, ориентируясь на отзывы других людей и на собственное местоположение. Классические картографические сервисы предоставляют справочную информацию, однако социальная составляющая в них вторична: отзывы разрознены, а возможность делиться «своими» местами с сообществом ограничена. В то же время специализированные геосоциальные сервисы либо закрыты для российского рынка, либо ориентированы на коммерческие заведения и не позволяют пользователю самостоятельно добавлять произвольные точки интереса.

Актуальность темы обусловлена растущей потребностью в приложениях, объединяющих карту, пользовательский контент и социальное взаимодействие в одном интерфейсе, а также распространением клиент-серверной архитектуры как стандарта разработки мобильных продуктов. Реализация подобного приложения требует решения комплекса инженерных задач: проектирования REST API, организации хранения геоданных, безопасной аутентификации и построения удобного интерфейса.

Объектом исследования является процесс разработки клиент-серверных мобильных приложений. Предметом исследования выступают методы и инструментальные средства проектирования и реализации мобильной социальной сети местоположений на платформе Android с собственным сервером.

Целью курсовой работы является разработка клиент-серверного мобильного приложения социальной сети местоположений, позволяющего пользователям добавлять места на карту, публиковать отзывы о них и просматривать активность других участников.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Описать предметную область и провести сравнительный анализ существующих аналогов.
2. Обосновать выбор инструментальных средств разработки.
3. Определить функциональные и нефункциональные требования к приложению.
4. Спроектировать пользовательский интерфейс и межэкранное взаимодействие.
5. Построить логическую модель приложения (диаграммы IDEF0 и классов) и модель базы данных.
6. Реализовать серверную часть на Kotlin с использованием фреймворка Ktor и СУБД PostgreSQL.
7. Реализовать клиентскую часть на Android с использованием Jetpack Compose по принципам Clean Architecture.
8. Провести тестирование разработанного приложения.

Пояснительная записка состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе анализируется предметная область, рассматриваются аналоги и обосновывается выбор инструментов. Во второй главе формулируются требования, проектируются интерфейс, логическая модель и база данных. Третья глава посвящена реализации клиентской и серверной частей, их интеграции и тестированию.

1 АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1 Описание предметной области мобильного приложения

Социальная сеть местоположений (геосоциальная сеть) — это разновидность социальной сети, в которой основной единицей контента является географическая точка — место, привязанное к координатам на карте. Вокруг мест формируется пользовательская активность: описания, отзывы, оценки, отметки о посещении. В отличие от классических социальных сетей, где лента строится вокруг людей и их публикаций, в геосоциальной сети лента и поиск строятся вокруг пространственного контекста: что находится рядом, что рекомендуют другие пользователи в конкретном районе, какие места популярны.

Ключевыми сущностями предметной области являются:

- пользователь — зарегистрированный участник сети, имеющий профиль (имя, электронная почта, описание «о себе») и выступающий автором мест и отзывов;
- место — точка интереса с названием, описанием, категорией (кафе, парк, смотровая площадка, спорт, культура и др.), географическими координатами и необязательным адресом;
- пост (отзыв) — текстовое сообщение пользователя, привязанное к конкретному месту, с необязательной оценкой по пятибалльной шкале;
- лента — хронологически упорядоченный список последних постов всех пользователей;
- карта — визуальное представление мест в виде маркеров с возможностью поиска в заданном радиусе от текущего местоположения.

Типовой сценарий использования выглядит следующим образом. Пользователь регистрируется по адресу электронной почты и паролю, после чего попадает в ленту, где видит отзывы других участников. Открыв карту, он видит

маркеры мест рядом с собой, может выполнить поиск по названию или добавить новое место, указав точку на карте, название, описание и категорию. Открыв карточку места, пользователь читает отзывы и публикует собственный отзыв с оценкой. В профиле он редактирует личные данные и просматривает список добавленных им мест.

С точки зрения архитектуры такое приложение является распределённой системой: данные о пользователях, местах и постах должны быть доступны всем участникам, поэтому они хранятся централизованно на сервере, а мобильный клиент лишь отображает их и отправляет изменения. Это определяет клиент-серверную модель взаимодействия: клиент обращается к серверу по протоколу HTTP, сервер обрабатывает запросы, применяет бизнес-правила и работает с базой данных [1].

Особенности предметной области, влияющие на проектирование:

- геопространственный поиск: необходимо находить места в радиусе от точки, что требует хранения координат и вычисления расстояний между точками на поверхности Земли;
- пользовательский контент: любой участник может создавать места и отзывы, следовательно, требуется аутентификация и разграничение прав (удалять место может только его автор);
- многопользовательский доступ: данные должны быть согласованы между всеми клиентами, поэтому источником истины является серверная база данных;
- мобильный контекст: интерфейс должен учитывать ограниченный размер экрана, сенсорное управление и работу с датчиком местоположения.

Таким образом, разрабатываемое приложение должно объединить функции карты, справочника мест и социальной ленты, обеспечивая при этом безопасную работу с пользовательскими данными.

1.2 Сравнительный анализ существующих аналогов

Для определения требований к разрабатываемому приложению были рассмотрены четыре сервиса, в которых реализованы функции работы с местами и пользовательскими отзывами: Swarm (Foursquare), Яндекс Карты, 2ГИС и Google Maps.

Swarm — приложение компании Foursquare, являющееся классическим представителем геосоциальных сетей [2]. Основной механикой является «чек-ин»: пользователь отмечает в заведении, зарабатывает баллы и видит отметки друзей. Приложение хорошо реализует социальную составляющую, однако ориентировано на коммерческие заведения из базы Foursquare, не поддерживает русскоязычный интерфейс в полном объеме, а часть функций недоступна в России.

Яндекс Карты — картографический сервис с развитой справочной базой организаций и системой отзывов [3]. Пользователи могут оставлять отзывы с оценками и фотографиями, однако добавление произвольных мест (например, любимой скамейки в парке) ограничено модерацией, а лента активности других пользователей отсутствует: сервис является справочником, а не социальной сетью.

2ГИС — справочник организаций с картой, ориентированный на поиск заведений и построение маршрутов [4]. Отзывы и оценки присутствуют, но социальные функции (профили, лента) отсутствуют; пользователь не может добавить собственную точку интереса, не являющуюся организацией.

Google Maps — глобальный картографический сервис с программой Local Guides, в рамках которой пользователи оставляют отзывы и фотографии [5]. Сервис обладает максимальным покрытием, но, как и Яндекс Карты, не предоставляет ленту активности и не позволяет создавать произвольные пользовательские места вне справочника организаций.

Результаты сравнения по ключевым критериям приведены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ аналогов

Критерий	Swarm	Яндекс Карты	2ГИС	Google Maps
Добавление произвольных мест пользователем	Частично	Нет	Нет	Нет
Отзывы и оценки	Да	Да	Да	Да
Лента активности пользователей	Да	Нет	Нет	Нет
Поиск мест рядом	Да	Да	Да	Да
Профиль пользователя	Да	Да	Частично	Да
Русскоязычный интерфейс	Частично	Да	Да	Да
Доступность в России	Ограничена	Да	Да	Частично
Открытое API для своего сервера	Нет	Нет	Нет	Нет

Анализ показывает, что ни один из аналогов не сочетает в полной мере три функции: свободное добавление мест пользователями, социальную ленту и русскоязычный интерфейс, доступный без ограничений. Кроме того, все рассмотренные сервисы являются закрытыми: разработчик не может использовать их серверную часть как основу собственного продукта. Это подтверждает целесообразность разработки собственного клиент-серверного приложения, в котором:

- пользователь может добавить любое место, указав точку на карте;
- вокруг мест формируется лента отзывов с оценками;
- реализован поиск мест рядом с текущим местоположением и по названию;
- интерфейс полностью русскоязычный;
- серверная часть принадлежит разработчику и может расширяться.

Из аналогов были заимствованы удачные решения: карта с маркерами и карточкой места (Яндекс Карты, 2ГИС), пятибалльная шкала оценок (Google Maps), хронологическая лента отзывов (Swarm).

1.3 Обоснование выбора инструментальных средств для разработки мобильного приложения

В соответствии с заданием на курсовую работу все части приложения реализуются на языке Kotlin. Kotlin является официально рекомендуемым языком разработки для Android, обладает статической типизацией, null-безопасностью, поддержкой корутин для асинхронного программирования и полной совместимостью с JVM, что позволяет использовать один язык и для клиента, и для сервера [6]. Общая архитектура разрабатываемой системы представлена на рисунке 1.1.

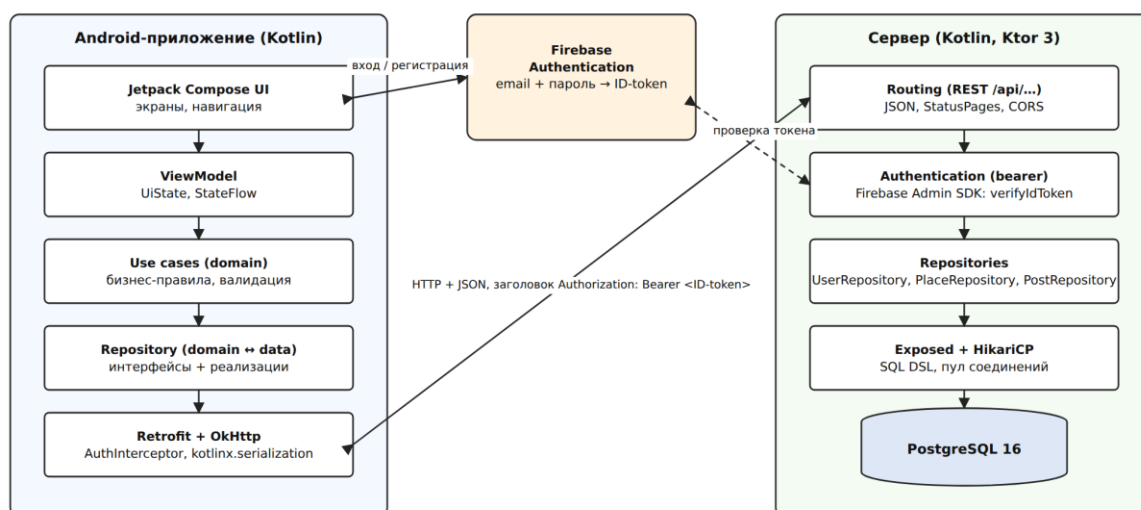


Рисунок 1.1 – Общая архитектура клиент-серверного приложения

Серверная часть. В качестве серверного фреймворка выбран Ktor версии 3 — асинхронный фреймворк от JetBrains, построенный на корутинах Kotlin [7]. По сравнению со Spring Boot Ktor обладает меньшим временем запуска и объёмом потребляемой памяти, не требует рефлексии и аннотаций, а маршрутизация описывается декларативно на Kotlin DSL. Для учебного проекта с ограниченным набором конечных точек Ktor позволяет сосредоточиться на логике приложения, а не на конфигурации фреймворка. Модульная система плагинов (ContentNegotiation, Authentication, StatusPages, CORS, CallLogging) подключает только необходимую функциональность.

Для работы с базой данных используется библиотека Exposed — типобезопасный SQL-фреймворк для Kotlin от JetBrains [8]. Exposed позволяет описывать таблицы как Kotlin-объекты, строить запросы через DSL с проверкой типов на этапе компиляции и автоматически создавать схему при старте. Пул соединений обеспечивает HikariCP — наиболее производительная реализация пула JDBC-соединений.

В качестве СУБД выбрана PostgreSQL 16 [9]. По сравнению с MySQL PostgreSQL строже соответствует стандарту SQL, поддерживает тип `double precision` для хранения координат без потери точности, транзакции с уровнем изоляции `REPEATABLE READ` и в перспективе позволяет подключить расширение PostGIS для полноценного геопространственного поиска. СУБД распространяется свободно и запускается одной командой через Docker Compose. Использование Firebase в качестве базы данных запрещено заданием; в разработанном приложении все данные хранятся исключительно в PostgreSQL.

Аутентификация. В соответствии с заданием вход выполняется по электронной почте и паролю через сервис Firebase Authentication [10]. Firebase используется исключительно как провайдер идентификации: он хранит учётные записи, проверяет пароли и выдаёт клиенту подписанный JWT-токен (ID-token). Сервер проверяет подпись токена с помощью Firebase Admin SDK и сопоставляет уникальный идентификатор пользователя (uid) с собственной таблицей `users`. Такой подход избавляет от необходимости самостоятельно хранить пароли и реализовывать восстановление доступа, при этом сервер и база данных остаются полностью собственными.

Клиентская часть. Интерфейс реализован на Jetpack Compose — современном декларативном UI-фреймворке Android [11]. По сравнению с XML-разметкой Compose сокращает объём кода, устраняет необходимость в `findViewById` и адаптерах, обеспечивает автоматическую перерисовку при изменении состояния и естественно сочетается с архитектурой MVVM через `StateFlow`. Библиотека `Navigation Compose` обеспечивает переходы между экранами, `Material 3` — единый визуальный стиль.

Для сетевого взаимодействия выбран Retrofit 2 совместно с OkHttp [12]. Retrofit описывает REST API в виде Kotlin-интерфейса с аннотациями, поддерживает suspend-функции и конвертеры `kotlinx.serialization`, что позволяет использовать одни и те же правила сериализации JSON на клиенте и сервере. Перехватчик OkHttp (Interceptor) централизованно добавляет токен авторизации в каждый запрос.

Внедрение зависимостей реализовано с помощью Hilt — библиотеки на базе Dagger, официально рекомендованной Google для Android [13]. Hilt автоматически создаёт ViewModel с зависимостями и упрощает подмену реализаций репозитория при тестировании. Отображение карты выполняется библиотекой Maps Compose, предоставляющей Google Maps SDK в виде composable-функций [14]; определение местоположения — сервисом Fused Location Provider.

Архитектура. Клиент построен по принципам Clean Architecture [15]: слой domain содержит модели, интерфейсы репозитория и сценарии использования (use case) и не зависит от фреймворков; слой data реализует репозитории через Retrofit и Firebase; слой presentation содержит ViewModel и composable-экраны. Направление зависимостей — от внешних слоёв к domain — упрощает тестирование и замену инфраструктурных компонентов. Сводка выбранных инструментов приведена в таблице 1.2.

Таблица 1.2 – Инструментальные средства разработки

Компонент	Инструмент, версия	Назначение
Язык	Kotlin 2.0	Клиент и сервер
Сервер	Ktor 3.0 (Netty)	REST API, плагины авторизации и сериализации
ORM / SQL	Exposed 0.55, HikariCP 5.1	Доступ к БД, пул соединений
СУБД	PostgreSQL 16	Хранение пользователей, мест, постов
Аутентификация	Firebase Authentication, Firebase Admin SDK 9.4	Вход по email и паролю, проверка токенов
UI клиента	Jetpack Compose (BOM 2024.10), Material 3	Декларативный интерфейс
Сеть клиента	Retrofit 2.11, OkHttp 4.12, kotlinx.serialization 1.7	Обращение к REST API
DI	Hilt 2.52	Внедрение зависимостей
Карта	Maps Compose 6.1, Play Services Location 21.3	Отображение карты, геолокация
Среда разработки	Android Studio, IntelliJ IDEA, Docker Desktop	Сборка и запуск

Выводы по главе. Анализ предметной области показал, что геосоциальная сеть объединяет карту, пользовательские места и ленту отзывов, а её реализация требует клиент-серверной архитектуры с централизованным хранилищем. Рассмотренные аналоги не сочетают свободное добавление мест, социальную ленту и открытость для разработчика, что обосновывает разработку собственного решения. Выбранный стек (Kotlin, Ktor, Exposed, PostgreSQL, Jetpack Compose, Retrofit, Hilt, Firebase Authentication) соответствует требованиям задания и современным практикам мобильной разработки.

2 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

2.1 Определение функциональных и нефункциональных требований

Функциональные требования описывают действия, которые система должна выполнять для пользователя. Они сформулированы на основе анализа предметной области и задания на курсовую работу и приведены в таблице 2.1.

Таблица 2.1 – Функциональные требования

Код	Требование	Описание
ФТ-1	Регистрация	Пользователь создаёт учётную запись, указав имя, адрес электронной почты и пароль длиной не менее 6 символов
ФТ-2	Авторизация	Пользователь входит в систему по электронной почте и паролю; сессия сохраняется между запусками приложения
ФТ-3	Выход из аккаунта	Пользователь завершает сессию, после чего приложение возвращается на экран входа
ФТ-4	Просмотр ленты	Приложение отображает последние отзывы всех пользователей с именем автора, названием места, текстом, оценкой и датой
ФТ-5	Просмотр карты	Приложение отображает места в виде маркеров на карте; при нажатии на маркер показывается название и категория
ФТ-6	Поиск мест	Пользователь ищет места по названию через поисковую строку и получает список мест в радиусе 10 км от своего местоположения
ФТ-7	Создание места	Пользователь выбирает точку на карте, вводит название, описание, категорию и адрес и сохраняет место на сервере
ФТ-8	Просмотр места	Приложение показывает подробную информацию о месте и список отзывов к нему
ФТ-9	Публикация отзыва	Пользователь оставляет текстовый отзыв о месте с необязательной оценкой от 1 до 5
ФТ-10	Профиль	Пользователь просматривает и редактирует имя и описание «о себе», видит список добавленных им мест
ФТ-11	Разграничение прав	Удалять место или отзыв может только его автор; неавторизованные запросы отклоняются сервером
ФТ-12	Обработка ошибок	При отсутствии связи с сервером или ошибке приложение показывает понятное сообщение и кнопку обновления

Взаимодействие пользователей и внешних систем с приложением представлено на диаграмме вариантов использования (рисунок 2.1). Выделены два актёра-человека — Гость (незарегистрированный пользователь) и Авторизованный пользователь — и две внешние системы: Firebase Authentication, проверяющая учётные данные, и сервер GeoSocial, хранящий данные о местах и отзывах. Отношения «include» показывают обязательные подсценарии (проверка учётных данных при входе, определение местоположения при поиске рядом), отношения «extend» — необязательные расширения (переход к карточке места из ленты или карты, публикация отзыва).

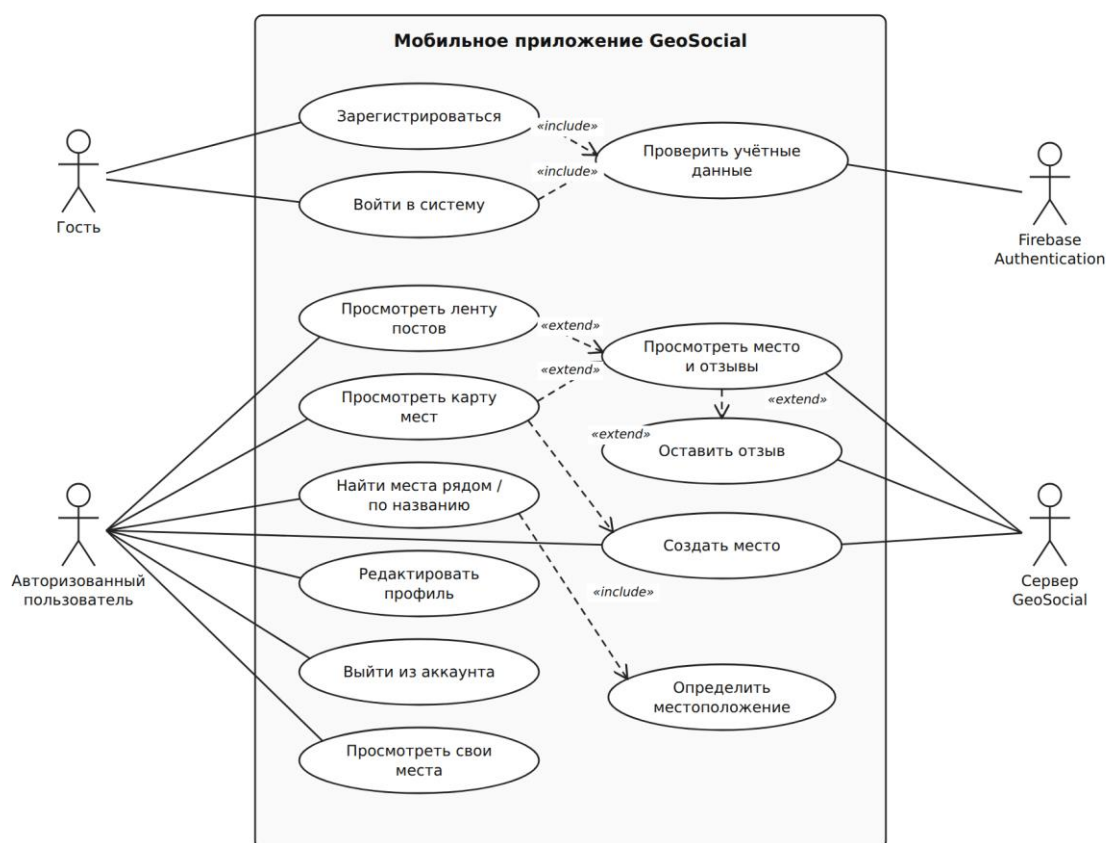


Рисунок 2.1 – Диаграмма вариантов использования

Нефункциональные требования определяют качественные характеристики системы и ограничения на её реализацию. Они приведены в таблице 2.2.

Таблица 2.2 – Нефункциональные требования

Код	Категория	Требование
НФТ-1	Платформа	Клиент работает на Android 7.0 (API 24) и выше; сервер запускается на JVM 17 в среде Windows, Linux или macOS
НФТ-2	Язык и стек	Клиент и сервер реализованы на Kotlin; интерфейс — Jetpack Compose; сервер — Ktor; СУБД — PostgreSQL
НФТ-3	Архитектура	Клиент построен по принципам Clean Architecture с разделением на слои presentation, domain и data
НФТ-4	Безопасность	Каждый запрос к API содержит подписанный ID-token; сервер отклоняет запросы без токена (401) и запрещённые действия (403)
НФТ-5	Производительность	Ответ сервера на запрос ленты или поиска мест не превышает 1 с при базе до 10 000 мест; в интерфейсе отображается индикатор загрузки
НФТ-6	Локализация	Весь пользовательский интерфейс и сообщения об ошибках представлены на русском языке
НФТ-7	Удобство	Интерфейс соответствует рекомендациям Material Design 3; основные разделы доступны через нижнюю панель навигации не более чем за одно нажатие
НФТ-8	Надёжность	Ошибки сервера возвращаются в едином JSON-формате; клиент корректно обрабатывает отсутствие сети и не завершается аварийно
НФТ-9	Сопровождаемость	Схема базы данных создаётся автоматически при запуске сервера; конфигурация задаётся через переменные окружения

Совокупность требований определяет минимально достаточный функционал социальной сети местоположений: аутентификация, карта с местами, создание мест, отзывы, лента и профиль. Требования ФТ-1 – ФТ-12 проверяются в ходе тестирования (раздел 3.3).

2.2 Проектирование пользовательского интерфейса

Макеты экранов разработаны в графическом редакторе Figma с использованием компонентов Material 3 и затем воспроизведены в Jetpack Compose. Цветовая схема построена на бирюзовом основном цвете (#00696D), обеспечивающем достаточный контраст с текстом; используется системный шрифт Roboto. Приложение содержит семь экранов: вход, регистрация, лента, карта, создание места, карточка места и профиль.

Экраны входа и регистрации представлены на рисунке 2.2. Экран входа (рисунок 2.2, а) содержит название приложения, поля «Email» и «Пароль»,

кнопку «Войти» и ссылку на регистрацию. При ошибке (неверный пароль, отсутствие сети) под полями отображается сообщение красным цветом; во время запроса кнопка блокируется и показывает индикатор загрузки. Экран регистрации (рисунок 2.2, б) дополняет форму полем «Имя» и подсказкой о минимальной длине пароля. Валидация выполняется на клиенте до отправки запроса: пустой email или пароль короче 6 символов приводят к сообщению без обращения к серверу.

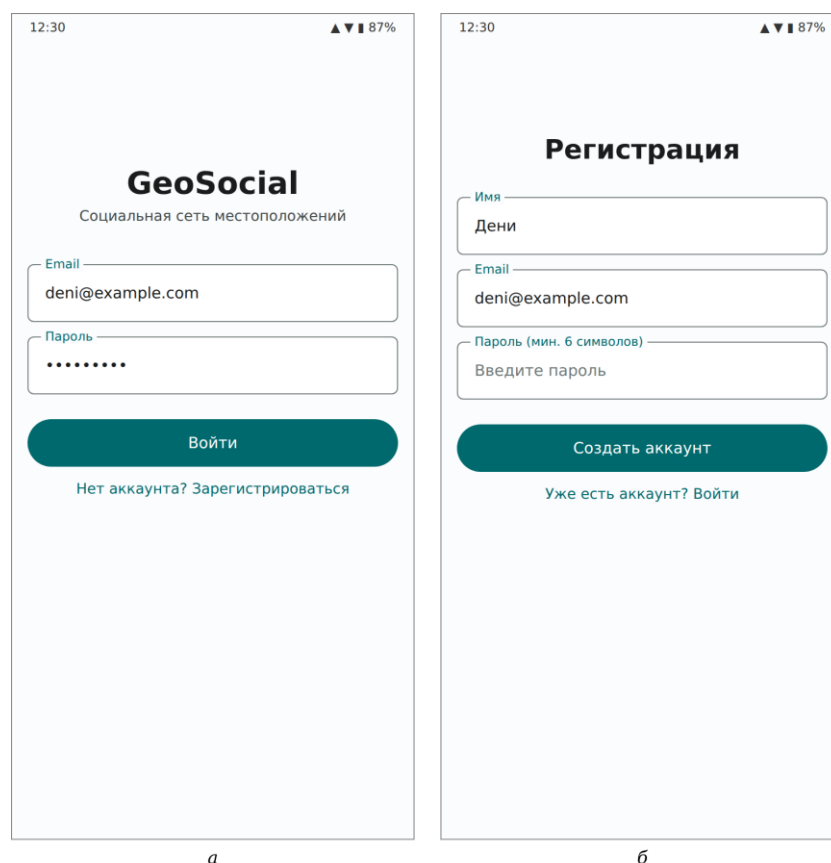


Рисунок 2.2 – Макеты экранов входа (а) и регистрации (б)

После входа пользователь попадает на главный экран с нижней панелью навигации из трёх вкладок: «Лента», «Карта», «Профиль». Макеты вкладок и экрана создания места показаны на рисунке 2.3. Лента (рисунок 2.3, а) представляет собой прокручиваемый список карточек отзывов. Каждая карточка содержит аватар-инициал автора, имя, название места (нажатие открывает карточку места), оценку, текст и дату. В верхней панели расположена кнопка обновления. При отсутствии постов показывается заглушка с приглашением добавить первое место, при ошибке — сообщение с текстом ошибки.

Экран карты (рисунок 2.3, б) занимает основную площадь картой Google Maps с маркерами мест. Над картой расположена поисковая строка с кнопкой «Найти», под картой — подсказка о добавлении места долгим нажатием. Плавающая кнопка «Добавить место» создаёт место в центре текущего окна карты. При нажатии на маркер появляется информационное окно с названием, категорией и числом отзывов; нажатие на окно открывает карточку места. При наличии разрешения на геолокацию карта центрируется на текущем положении пользователя и запрашивает места в радиусе 10 км.

Экран создания места (рисунок 2.3, в) содержит уменьшенную карту, на которой нажатием уточняется точка, текстовые поля «Название», «Описание», «Адрес», выбор категории с помощью чипов и кнопку «Сохранить место». Координаты выбранной точки отображаются под картой. После успешного сохранения приложение возвращается на карту.

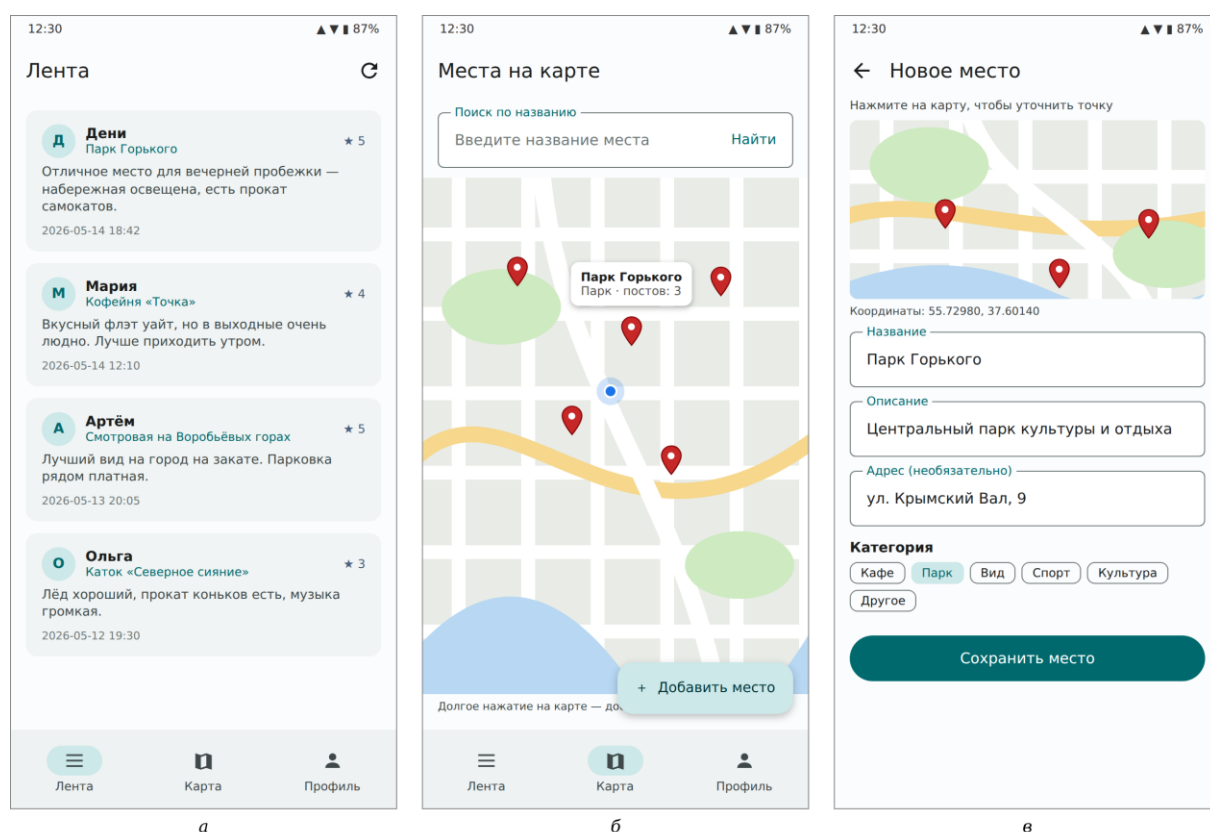


Рисунок 2.3 – Макеты экранов ленты (а), карты (б) и создания места (в)

Карточка места (рисунок 2.4, а) показывает название, категорию, описание, адрес, координаты и автора, форму нового отзыва с выбором оценки и список существующих отзывов. Кнопка «Опубликовать» активна только при непустом

тексте; после отправки новый отзыв появляется в начале списка без перезагрузки экрана. Экран профиля (рисунок 2.4, б) отображает имя, электронную почту и описание пользователя, кнопку «Редактировать», переводящую карточку в режим ввода, а также список мест, добавленных пользователем. Кнопка в верхней панели выполняет выход из аккаунта.

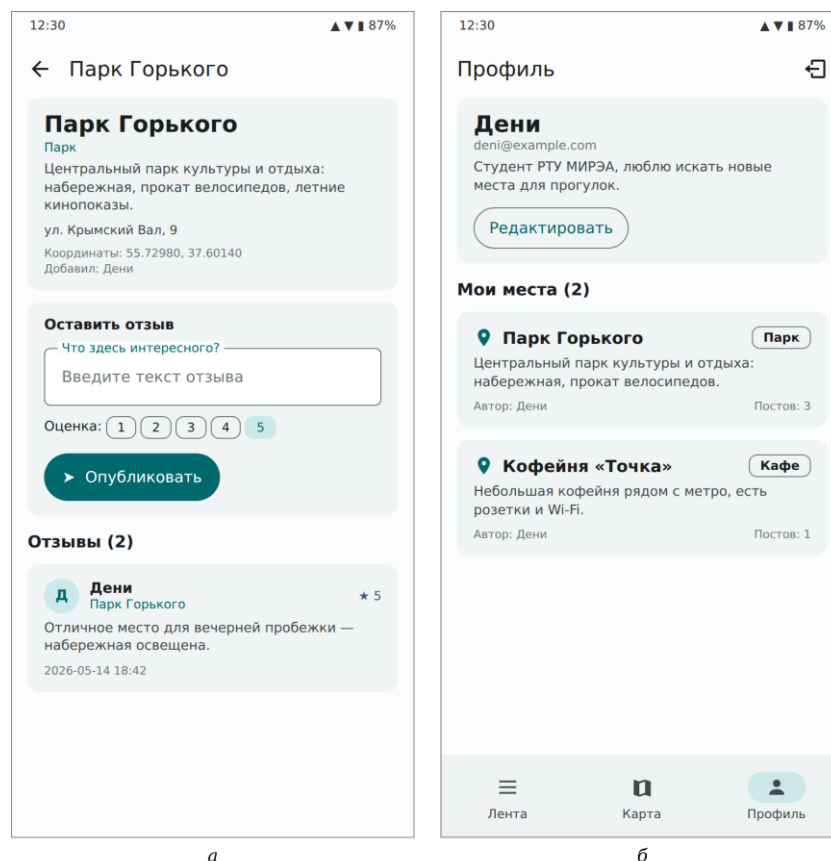


Рисунок 2.4 – Макеты экранов места (а) и профиля (б)

Межэкранное взаимодействие представлено на рисунке 2.5. Экраны входа и регистрации образуют граф аутентификации; после успешного входа приложение переключается на граф главного экрана. Три вкладки доступны из нижней панели, при этом состояние каждой вкладки сохраняется при переключении. Карточка места открывается из ленты, карты и профиля; экран создания места — только из карты. Выход из аккаунта возвращает пользователя на экран входа с очисткой стека навигации.

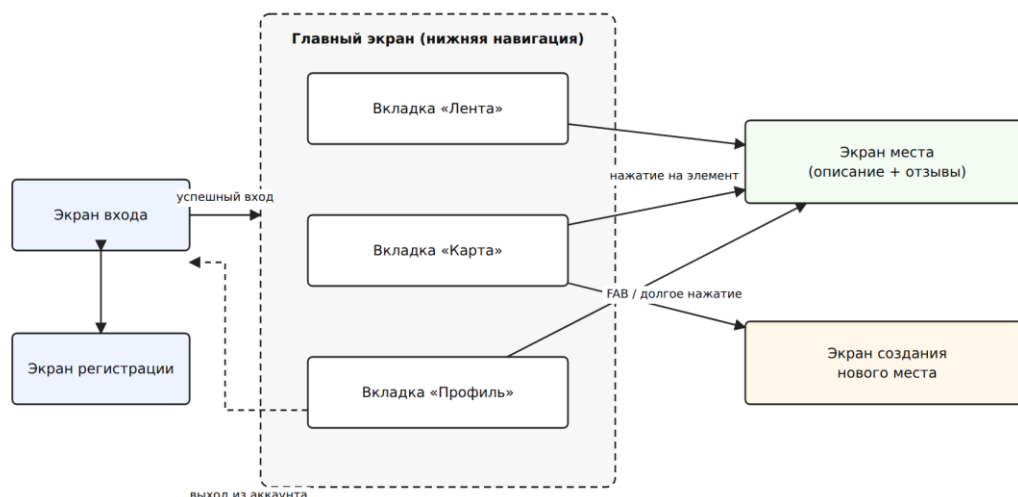


Рисунок 2.5 – Схема межэкранного взаимодействия

Такая структура обеспечивает выполнение требования НФТ-7: любой основной раздел доступен за одно нажатие, а вложенные экраны имеют кнопку «Назад» в верхней панели.

2.3 Логическая модель приложения

2.3.1 Проектирование мобильного приложения

Функциональная модель системы построена в нотации IDEF0 [16]. Контекстная диаграмма A-0 (рисунок 2.6) представляет систему как единый процесс «Обеспечить обмен информацией о местах в социальной сети местоположений». Входами являются учётные данные пользователя, данные о месте, текст и оценка отзыва, поисковый запрос. Выходы — лента постов, карта с местами, профиль пользователя и результат операции. Управляющими воздействиями выступают требования задания, правила валидации данных и политика Firebase Authentication; механизмами — мобильное приложение, сервер Ktor, СУБД PostgreSQL и сервис Firebase Authentication.



Узел: A-0

Контекстная диаграмма

Рисунок 2.6 – Контекстная диаграмма IDEF0 (узел A-0)

Декомпозиция контекстной диаграммы (рисунок 2.7) выделяет четыре функциональных блока. Блок A1 «Аутентифицировать пользователя» принимает учётные данные и формирует ID-токен, который служит входом для остальных блоков и выходом «Профиль». Блок A2 «Управлять местами» обрабатывает создание, поиск и удаление мест. Блок A3 «Публиковать отзывы» принимает текст и оценку, связывает их с идентификатором места. Блок A4 «Формировать ленту и результаты поиска» агрегирует данные мест и постов по поисковому запросу и координатам, формируя ленту и карту. Все блоки, кроме A1, выполняются мобильным приложением совместно с сервером и СУБД.

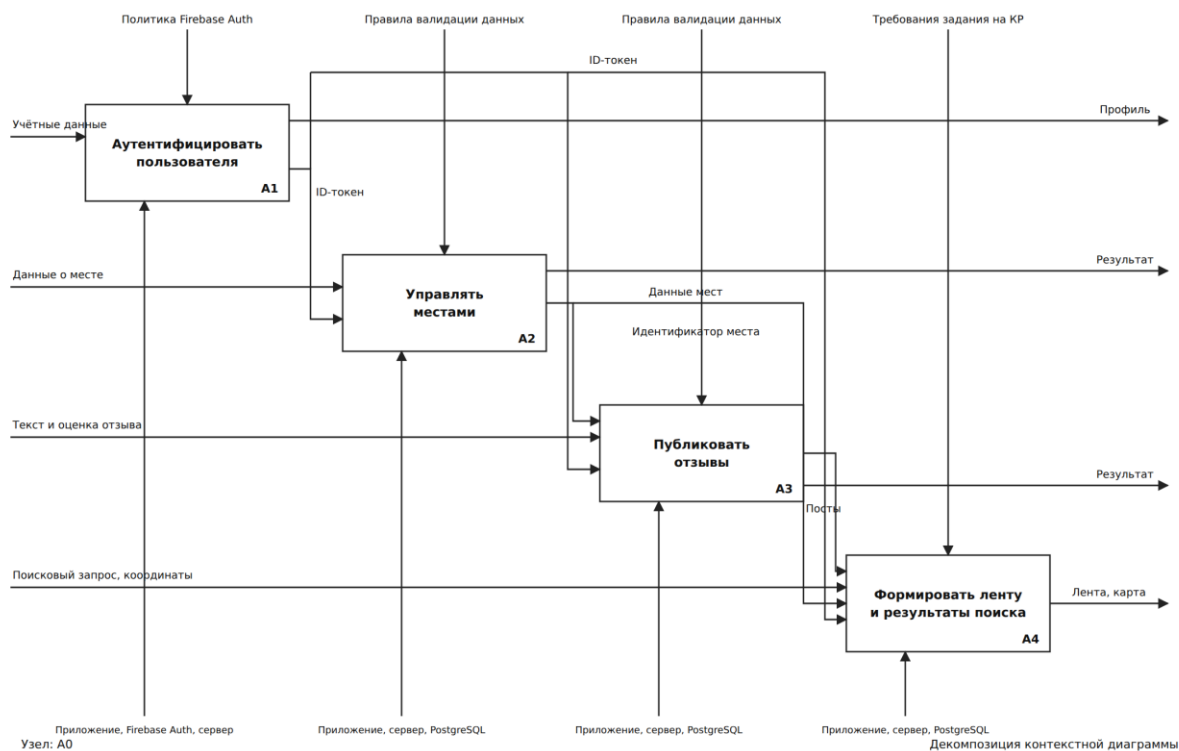


Рисунок 2.7 – Декомпозиция диаграммы IDEF0 (узел A0)

Статическая структура клиентской части представлена диаграммой классов на рисунке 2.8. Диаграмма отражает три слоя Clean Architecture. В слое presentation находятся ViewModel-классы, каждый из которых хранит состояние экрана в виде неизменяемого data-класса (например, FeedUiState) и публикует его через StateFlow. ViewModel обращаются к сценариям использования слоя domain (SignInUseCase, GetPlacesUseCase, CreatePlaceUseCase, AddPostUseCase и др.), которые содержат проверки бизнес-правил и делегируют работу интерфейсам репозитория AuthRepository, PlaceRepository, PostRepository и UserRepository. Модели User, Place и Post не зависят от способа получения данных. Слой data содержит реализации репозитория, Retrofit-интерфейс GeoSocialApi, источник данных FirebaseAuthDataSource и перехватчик AuthInterceptor. Реализации зависят от интерфейсов domain (отношение реализации), а не наоборот, что соответствует принципу инверсии зависимостей.

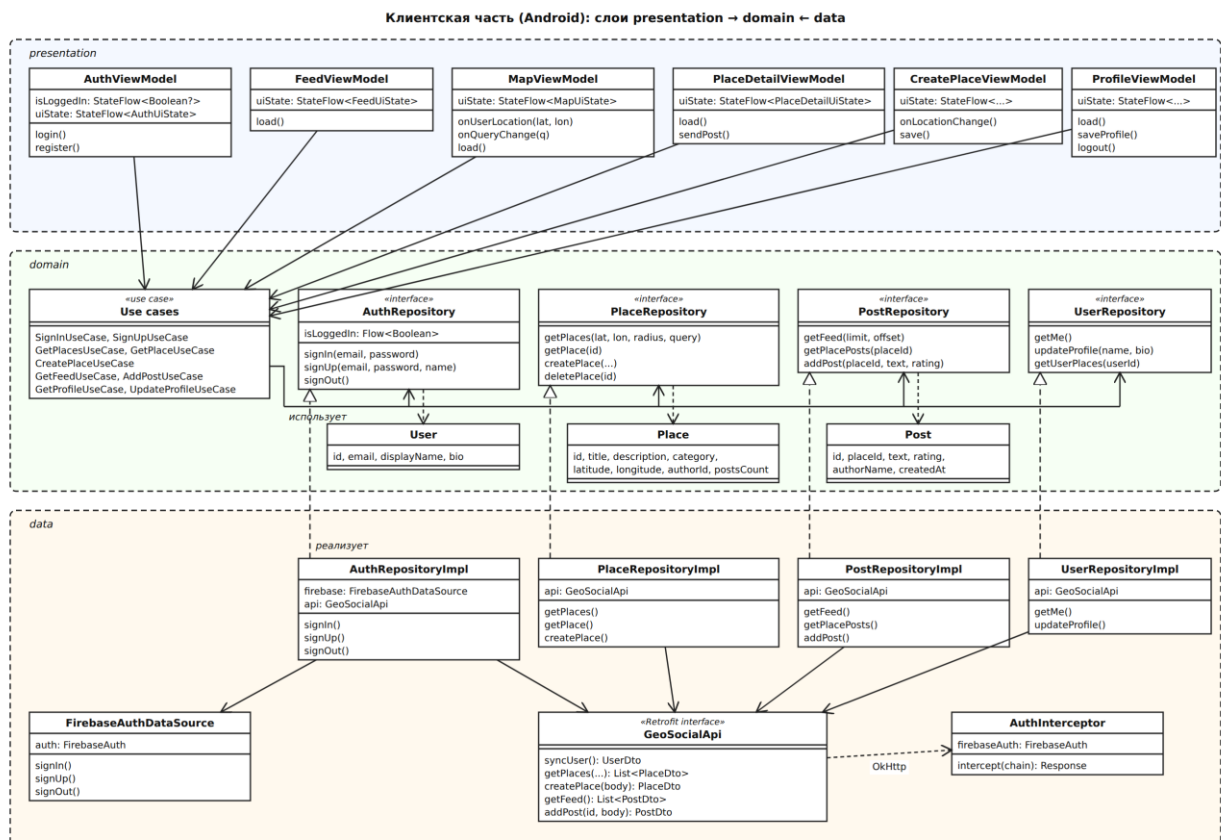


Рисунок 2.8 – Диаграмма классов клиентской части

Структура серверной части показана на рисунке 2.9. Точкой входа является функция `Application.module()`, которая инициализирует базу данных через объект `DatabaseFactory`, создаёт сервис проверки токенов `FirebaseAuthService` и репозитории, после чего подключает плагины и маршруты. Маршруты сгруппированы по ресурсам (`userRoutes`, `placeRoutes`, `postRoutes`) и обращаются к репозиториям `UserRepository`, `PlaceRepository` и `PostRepository`, инкапсулирующим запросы к таблицам `Users`, `Places` и `Posts`, описанным средствами `Exposed`. Класс `UserPrincipal` представляет авторизованного пользователя текущего запроса и формируется плагином аутентификации на основе результата проверки токена.

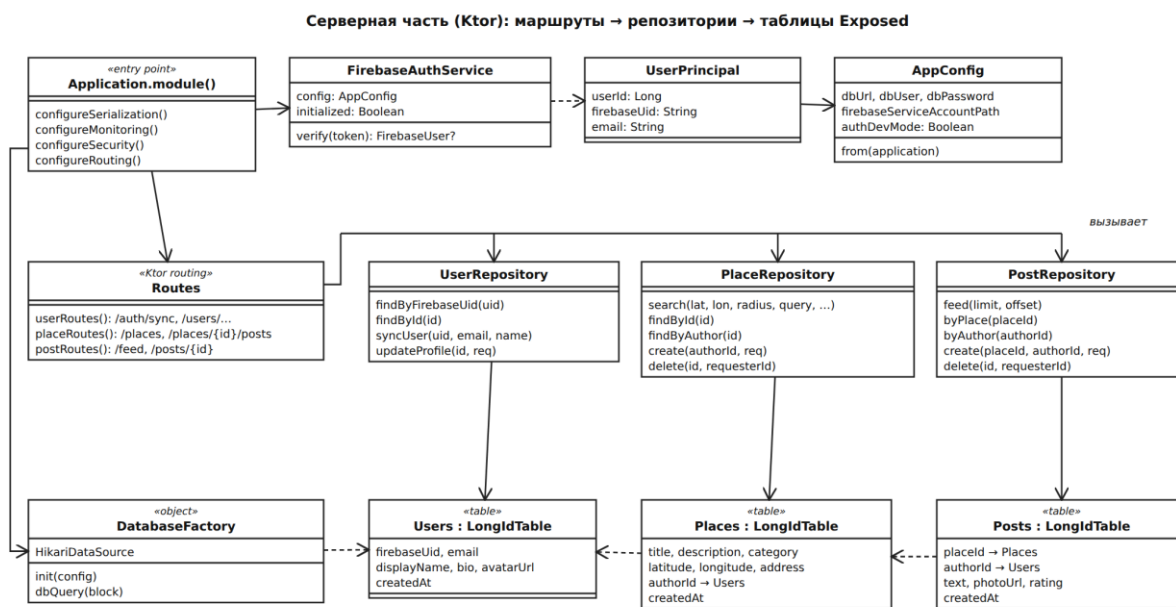


Рисунок 2.9 – Диаграмма классов серверной части

2.3.2 Проектирование базы данных

Логическая модель базы данных содержит три сущности — users, places и posts, связанные отношениями «один ко многим» (рисунок 2.10). Пользователь может создать много мест и много постов; место содержит много постов; каждый пост принадлежит ровно одному месту и одному автору. Внешние ключи объявлены с правилом ON DELETE CASCADE: при удалении места удаляются его посты, при удалении пользователя — его места и посты. Модель находится в третьей нормальной форме: все неключевые атрибуты зависят только от первичного ключа, повторяющиеся группы отсутствуют.

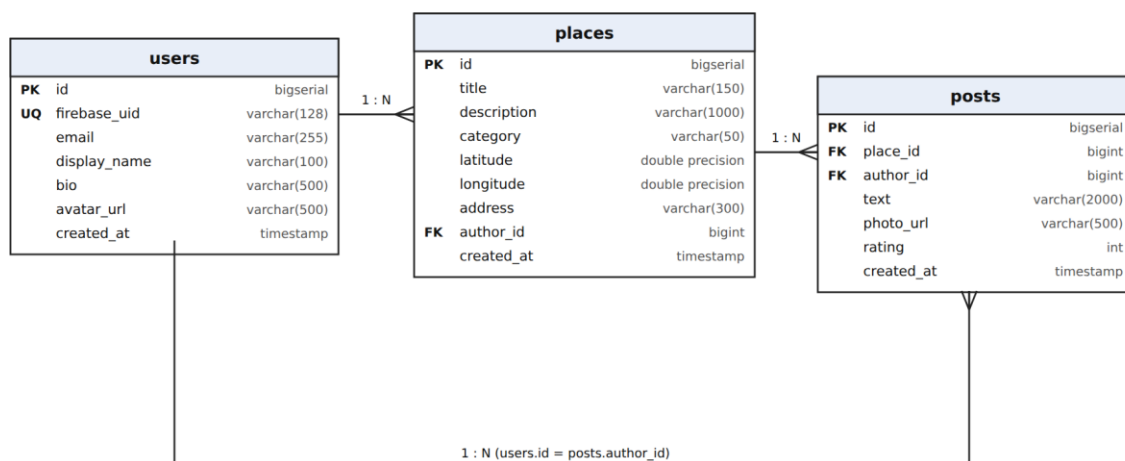


Рисунок 2.10 – Логическая модель базы данных

Таблица `users` (таблица 2.3) хранит профили пользователей. Поле `firebase_uid` содержит уникальный идентификатор, выданный Firebase Authentication, и служит связующим звеном между внешней системой аутентификации и собственной базой данных. Пароли в базе не хранятся.

Таблица 2.3 – Структура таблицы `users`

Поле	Тип	Ключ	Описание
<code>id</code>	<code>bigserial</code>	PK	Суррогатный первичный ключ
<code>firebase_uid</code>	<code>varchar(128)</code>	UNIQUE	Идентификатор пользователя в Firebase
<code>email</code>	<code>varchar(255)</code>	—	Адрес электронной почты
<code>display_name</code>	<code>varchar(100)</code>	—	Отображаемое имя
<code>bio</code>	<code>varchar(500)</code>	—	Описание «о себе» (необязательно)
<code>avatar_url</code>	<code>varchar(500)</code>	—	Ссылка на аватар (необязательно)
<code>created_at</code>	<code>timestamp</code>	—	Дата и время регистрации

Таблица `places` (таблица 2.4) хранит места. Координаты представлены типом `double precision`, что обеспечивает точность порядка сантиметров. Для ускорения поиска по прямоугольной области создаётся составной индекс по полям `latitude` и `longitude`.

Таблица 2.4 – Структура таблицы *places*

Поле	Тип	Ключ	Описание
id	bigserial	PK	Первичный ключ
title	varchar(150)	–	Название места
description	varchar(1000)	–	Описание
category	varchar(50)	–	Категория (Кафе, Парк, Вид, Спорт, Культура, Другое)
latitude	double precision	–	Широта, градусы
longitude	double precision	–	Долгота, градусы
address	varchar(300)	–	Адрес (необязательно)
author_id	bigint	FK → users.id	Автор места
created_at	timestamp	–	Дата и время создания

Таблица *posts* (таблица 2.5) хранит отзывы. Оценка *rating* допускает значение NULL, если пользователь не поставил оценку; допустимый диапазон 1–5 контролируется на сервере. Поле *photo_url* зарезервировано для дальнейшего расширения — загрузки фотографий.

Таблица 2.5 – Структура таблицы *posts*

Поле	Тип	Ключ	Описание
id	bigserial	PK	Первичный ключ
place_id	bigint	FK → places.id	Место, к которому относится отзыв
author_id	bigint	FK → users.id	Автор отзыва
text	varchar(2000)	–	Текст отзыва
photo_url	varchar(500)	–	Ссылка на фотографию (зарезервировано)
rating	int	–	Оценка от 1 до 5 (необязательно)
created_at	timestamp	–	Дата и время публикации

Выводы по главе. Сформулированы двенадцать функциональных и девять нефункциональных требований, охватывающих аутентификацию, работу с картой, местами, отзывами и профилем. Спроектированы макеты семи экранов и схема навигации с нижней панелью. Построены функциональная модель IDEF0, диаграммы классов клиента и сервера, отражающие слои Clean Architecture, и логическая модель базы данных из трёх таблиц в третьей нормальной форме. Полученные проектные решения являются основой для реализации, описанной в третьей главе.

3 РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ

3.1 Разработка клиентской части мобильного приложения

Клиентская часть реализована как Android-проект в среде Android Studio на языке Kotlin 2.0 с использованием Jetpack Compose [17]. Пакетная структура проекта повторяет слои Clean Architecture и рекомендации Google по архитектуре приложений [18]: пакет domain содержит модели, интерфейсы репозитория и сценарии использования; пакет data — Retrofit-интерфейс, DTO, мапперы, реализации репозитория и источник данных Firebase; пакет presentation — ViewModel и composable-экраны, сгруппированные по функциям (auth, feed, map, place, createplace, profile, navigation, components); пакет di — модули Hilt.

Модели предметной области объявлены как неизменяемые data-классы без зависимостей от фреймворков. Сценарии использования оформлены как классы с оператором invoke, что позволяет вызывать их как функции. Пример сценария входа с проверкой входных данных приведён в листинге 3.1.

Листинг 3.1 – Сценарий использования SignInUseCase

```
class SignInUseCase @Inject constructor(
    private val repository: AuthRepository
) {
    suspend operator fun invoke(email: String, password: String): Result<Unit> {
        if (email.isBlank() || password.isBlank()) {
            return Result.failure(
                IllegalArgumentException("Заполните email и пароль")
            )
        }
        return repository.signIn(email.trim(), password)
    }
}
```

Результаты операций передаются в виде стандартного типа Result<T>, что позволяет единообразно обрабатывать успех и ошибку в ViewModel без исключений. Сетевые ошибки преобразуются в понятные пользователю сообщения функцией safeApiCall: код 401 сопровождается текстом «Требуется вход в приложение», 403 — «Нет прав на это действие», исключение IOException — «Нет связи с сервером».

Состояние каждого экрана описано data-классом. ViewModel хранит его в MutableStateFlow и обновляет через copy(), а composable-функция подписывается на поток вызовом collectAsState(). Такой подход (однонаправленный поток данных) гарантирует, что интерфейс всегда отражает актуальное состояние. Фрагмент ViewModel ленты приведён в листинге 3.2.

Листинг 3.2 – Загрузка ленты в FeedViewModel

```
data class FeedUiState(
    val posts: List<Post> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null
)

@HiltViewModel
class FeedViewModel @Inject constructor(
    private val getFeed: GetFeedUseCase
) : ViewModel() {
    private val _uiState = MutableStateFlow(FeedUiState())
    val uiState: StateFlow<FeedUiState> = _uiState.asStateFlow()

    init { load() }

    fun load() {
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(isLoading = true, error = null)
            getFeed()
                .onSuccess {
                    _uiState.value = FeedUiState(posts = it, isLoading = false)
                }
                .onFailure {
                    _uiState.value = _uiState.value.copy(
                        isLoading = false, error = it.message
                    )
                }
        }
    }
}
```

Экран ленты (FeedScreen) отображает одно из четырёх состояний: индикатор загрузки, сообщение об ошибке, заглушку при пустом списке или список карточек LazyColumn. Карточки постов и мест вынесены в переиспользуемые composable-функции PostCard и PlaceCard в пакете components.

Экран карты использует библиотеку Maps Compose. Composable-функция GoogleMap принимает состояние камеры и обработчик долгого нажатия; внутри неё для каждого места создаётся маркер Marker с заголовком и подписью, а обработчик onInfoWindowClick открывает карточку места. Разрешение на

геолокацию запрашивается через `rememberLauncherForActivityResult`; при его получении `FusedLocationProviderClient` возвращает последнее известное положение, камера центрируется на нём, а `ViewModel` запрашивает места в радиусе 10 км. Поисковая строка связана с полем `query` состояния; кнопка «Найти» повторяет запрос с текущим текстом.

Экран создания места получает начальные координаты через аргументы навигации (`lat`, `lng`), которые `ViewModel` читает из `SavedStateHandle`. Нажатие на встроенную карту перемещает маркер и обновляет координаты в состоянии; категория выбирается из фиксированного списка `PlaceCategories`. Сценарий `CreatePlaceUseCase` проверяет непустоту названия и описания до обращения к серверу.

Аутентификация выполняется классом `FirebaseAuthDataSource`, оборачивающим `FirebaseAuth`: методы `signInWithEmailAndPassword` и `createUserWithEmailAndPassword` вызываются через расширение `await()` из библиотеки `kotlinx-coroutines-play-services`. Состояние входа публикуется как `Flow<Boolean>` через `AuthStateListener`, поэтому корневая `composable`-функция `AppNavigation` автоматически переключается между графом аутентификации и главным графом при входе и выходе (листинг 3.3).

Листинг 3.3 – Переключение графов навигации по состоянию входа

```
@Composable
fun AppNavigation() {
    val authViewModel: AuthViewModel = hiltViewModel()
    val isLoggedIn by authViewModel.isLoggedIn.collectAsState()
    when (isLoggedIn) {
        null -> Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center)
    {
        CircularProgressIndicator()
    }
        false -> AuthNavHost(authViewModel)
        true -> MainNavHost()
    }
}
```

Главный граф `MainNavHost` содержит `Scaffold` с нижней панелью `NavigationBar`, отображаемой только на трёх корневых вкладках, и `NavHost` с маршрутами `feed`, `map`, `profile`, `place/{placeId}` и `create_place?lat={lat}&lng={lng}`.

При переключении вкладок используются параметры `launchSingleTop`, `saveState` и `restoreState`, что сохраняет состояние прокрутки и введённого текста.

Внедрение зависимостей настроено в двух модулях Hilt: `AppModule` предоставляет `FirebaseAuth`, `Json`, `OkHttpClient`, `Retrofit` и `GeoSocialApi`, а `RepositoryModule` связывает интерфейсы репозитория с их реализациями через `@Binds`. Все `ViewModel` помечены `@HiltViewModel` и получают сценарии использования через конструктор, что исключает ручное создание объектов и упрощает подмену реализаций при тестировании.

3.2 Разработка серверной части мобильного приложения

3.2.1 Описание основной логики сервера

Серверная часть представляет собой Gradle-проект на Kotlin с фреймворком Ktor 3.0 и движком Netty. Точка входа — функция `main`, запускающая `EngineMain` с конфигурацией из файла `application.conf`, где задаются порт, параметры подключения к базе данных и путь к ключу сервисного аккаунта Firebase; каждый параметр может быть переопределён переменной окружения (`PORT`, `DB_URL`, `DB_USER`, `DB_PASSWORD`, `FIREBASE_SERVICE_ACCOUNT`). Модуль приложения инициализирует базу данных, создаёт репозитории и подключает плагины (листинг 3.4).

Листинг 3.4 – Модуль приложения Ktor

```
fun Application.module() {
    val config = AppConfig.from(this)
    DatabaseFactory.init(config)

    val userRepository = UserRepository()
    val placeRepository = PlaceRepository()
    val postRepository = PostRepository()
    val authService = FirebaseAuthService(config)

    configureSerialization()
    configureMonitoring()
    configureSecurity(authService, userRepository)
    configureRouting(userRepository, placeRepository, postRepository)
}
```

Плагин `ContentNegotiation` с `kotlinx.serialization` отвечает за преобразование DTO в JSON и обратно. Плагин `StatusPages` перехватывает исключения:

собственные исключения `AppException` (`NotFound`, `BadRequest`, `Unauthorized`, `Forbidden`) преобразуются в ответы с кодами 404, 400, 401, 403 и телом `ErrorResponse`, содержащим код и сообщение на русском языке; любые другие исключения возвращают 500. Плагины `CallLogging` и `CORS` обеспечивают журналирование запросов и доступ из браузерных клиентов.

Аутентификация реализована плагином `Authentication` со схемой `bearer`. При каждом запросе к защищённому маршруту сервис `FirebaseAuthService` проверяет ID-токен через `Firebase Admin SDK`; при успешной проверке репозиторий пользователей находит или создаёт запись по `firebase_uid`, и в контекст запроса помещается объект `UserPrincipal`. Для отладки без `Firebase` предусмотрен `dev`-режим, в котором принимаются токены вида `dev:<uid>` (листинг 3.5).

Листинг 3.5 – Настройка *bearer*-аутентификации

```
install(Authentication) {
  bearer(AUTH_FIREBASE) {
    realm = "geosocial"
    authenticate { tokenCredential ->
      val firebaseUser = authService.verify(tokenCredential.token)
      ?: return@authenticate null
      val user = userRepository.syncUser(
        firebaseUid = firebaseUser.uid,
        email = firebaseUser.email,
        displayName = firebaseUser.displayName
      )
      UserPrincipal(
        user.id, firebaseUser.uid, user.email, user.displayName
      )
    }
  }
}
```

Маршруты объединены под префиксом `/api` и сгруппированы по ресурсам. Каждый обработчик извлекает параметры запроса, вызывает метод репозитория и отправляет результат. Полный перечень конечных точек приведён в таблице 3.1.

Таблица 3.1 – Конечные точки *REST API*

Метод	Путь	Назначение
GET	/api/health	Проверка доступности сервера
POST	/api/auth/sync	Создание или получение профиля после входа через Firebase

Метод	Путь	Назначение
GET	/api/users/me	Профиль текущего пользователя
PATCH	/api/users/me	Изменение имени, описания, аватара
GET	/api/users/{id}	Профиль пользователя по идентификатору
GET	/api/users/{id}/places	Места, созданные пользователем
GET	/api/users/{id}/posts	Посты пользователя
GET	/api/places	Список мест; параметры lat, lon, radius, query, category, limit, offset
GET	/api/places/{id}	Информация о месте
POST	/api/places	Создание места
DELETE	/api/places/{id}	Удаление своего места
GET	/api/places/{id}/posts	Отзывы к месту
POST	/api/places/{id}/posts	Публикация отзыва
GET	/api/feed	Общая лента постов; параметры limit, offset
DELETE	/api/posts/{id}	Удаление своего поста

Наиболее сложная логика сосредоточена в методе `PlaceRepository.search`, реализующем поиск мест рядом. Точное вычисление расстояния для всех записей таблицы было бы неэффективно, поэтому применяется двухэтапный алгоритм: сначала в SQL-запросе отбираются места внутри ограничивающего прямоугольника, стороны которого вычисляются из радиуса (1° широты ≈ 111 км, 1° долготы $\approx 111 \cdot \cos \varphi$ км), затем для отобранных записей вычисляется точное расстояние по формуле гаверсинусов (листинг 3.6), результаты фильтруются по радиусу и сортируются по возрастанию расстояния.

Листинг 3.6 – Вычисление расстояния по формуле гаверсинусов

```
fun distanceKm(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Double {
    val dLat = Math.toRadians(lat2 - lat1)
    val dLon = Math.toRadians(lon2 - lon1)
    val a = sin(dLat / 2).pow(2) +
            cos(Math.toRadians(lat1)) * cos(Math.toRadians(lat2)) *
            sin(dLon / 2).pow(2)
    return EARTH_RADIUS_KM * 2 * atan2(sqrt(a), sqrt(1 - a))
}
```

Репозитории также реализуют проверки бизнес-правил: название места не может быть пустым, координаты должны находиться в допустимых диапазонах, оценка отзыва — в диапазоне от 1 до 5; удаление места или поста разрешено только автору, иначе выбрасывается исключение `Forbidden`. Количество постов

для каждого места вычисляется одним запросом с группировкой, что исключает проблему N+1 при формировании списка.

3.2.2 Подключение базы данных к серверу

База данных PostgreSQL 16 разворачивается через Docker Compose: файл `docker-compose.yml` описывает контейнер `postgres:16-alpine` с именем базы, пользователем и паролем `geosocial` и постоянным томом для данных. Подключение сервера к базе выполняет объект `DatabaseFactory` (листинг 3.7). `HikariCP` [20] создаёт пул из десяти соединений с драйвером `org.postgresql.Driver` и уровнем изоляции `REPEATABLE READ`; `Exposed` подключается к пулу и при старте создаёт отсутствующие таблицы методом `SchemaUtils.createMissingTablesAndColumns`, что избавляет от ручного выполнения SQL-скриптов при развёртывании.

Листинг 3.7 – Инициализация подключения к базе данных

```
object DatabaseFactory {
  fun init(config: AppConfig) {
    val hikariConfig = HikariConfig().apply {
      driverClassName = "org.postgresql.Driver"
      jdbcUrl = config.dbUrl
      username = config.dbUser
      password = config.dbPassword
      maximumPoolSize = 10
      isAutoCommit = false
      transactionIsolation = "TRANSACTION_REPEATABLE_READ"
      validate()
    }
    Database.connect(HikariDataSource(hikariConfig))
    transaction {
      SchemaUtils.createMissingTablesAndColumns(Users, Places, Posts)
    }
  }

  suspend fun <T> dbQuery(block: suspend Transaction.() -> T): T =
    newSuspendedTransaction(Dispatchers.IO) { block() }
}
```

Таблицы описаны как объекты, наследующие `LongIdTable` (листинг 3.8). Внешние ключи объявляются функцией `reference` с правилом каскадного удаления; уникальность `firebase_uid` обеспечивается индексом `uniqueIndex`. Функция `dbQuery` выполняет блок кода в приостанавливаемой транзакции на

диспетчере IO с использованием корутин [19], поэтому обработчики Ktor не блокируют потоки сервера во время обращения к базе.

Листинг 3.8 – Описание таблицы places средствами Exposed

```
object Places : LongIdTable("places") {
    val title = varchar("title", 150)
    val description = varchar("description", 1000)
    val category = varchar("category", 50)
    val latitude = double("latitude")
    val longitude = double("longitude")
    val address = varchar("address", 300).nullable()
    val authorId = reference(
        "author_id", Users, onDelete = ReferenceOption.CASCADE
    )
    val createdAt = datetime("created_at")
}
```

Запросы строятся типобезопасным DSL: соединение таблиц places и users выполняется через innerJoin, условия формируются операторами eq, like, greaterEq, lessEq, а результат преобразуется в DTO функциями toDto. Благодаря этому ошибки в именах столбцов и типах выявляются на этапе компиляции, а не во время выполнения.

3.2.3 Подключение сервера к клиентской части мобильного приложения

Взаимодействие клиента и сервера организовано по протоколу HTTP с обменом сообщениями в формате JSON. Базовый адрес сервера задан в поле BuildConfig.BASE_URL и по умолчанию равен http://10.0.2.2:8080/ — адресу хост-машины из эмулятора Android; для физического устройства указывается IP-адрес компьютера в локальной сети. Поскольку в учебном проекте используется незашифрованное соединение, в манифесте разрешён cleartext-трафик.

Retrofit-интерфейс GeoSocialApi описывает конечные точки сервера аннотациями @GET, @POST, @PATCH, @DELETE с параметрами @Path, @Query и @Body; все методы объявлены как suspend и возвращают DTO, структура которых совпадает с серверными DTO. Экземпляр Retrofit создаётся в модуле AppModule с конвертером kotlinx.serialization, у которого включён параметр ignoreUnknownKeys, что делает клиент устойчивым к добавлению новых полей на сервере.

Ключевым элементом интеграции является перехватчик `AuthInterceptor` (листинг 3.9). Перед отправкой каждого запроса он получает актуальный ID-токен текущего пользователя Firebase (метод `getIdToken` автоматически обновляет истёкший токен) и добавляет заголовок `Authorization: Bearer <token>`. Таким образом, ни один слой приложения выше `data` не знает о механизме авторизации.

Листинг 3.9 – Перехватчик, добавляющий токен в запросы

```
@Singleton
class AuthInterceptor @Inject constructor(
    private val firebaseAuth: FirebaseAuth
) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = runBlocking {
            firebaseAuth.currentUser?.let { user ->
                runCatching { user.getIdToken(false).await().token }
                    .getOrNull()
            }
        }
        val request = chain.request().newBuilder()
            .apply { token?.let { header("Authorization", "Bearer $it") } }
            .build()
        return chain.proceed(request)
    }
}
```

Последовательность взаимодействия при входе показана на рисунке 3.1. Пользователь вводит учётные данные; приложение обращается к `Firebase Authentication` и получает ID-токен; затем вызывает `POST /api/auth/sync` с токеном в заголовке. Сервер проверяет подпись токена через `Firebase Admin SDK`, находит или создаёт запись в таблице `users` и возвращает профиль. Все последующие запросы (лента, места, отзывы) проходят ту же проверку, но запись пользователя уже существует, поэтому выполняется только чтение.

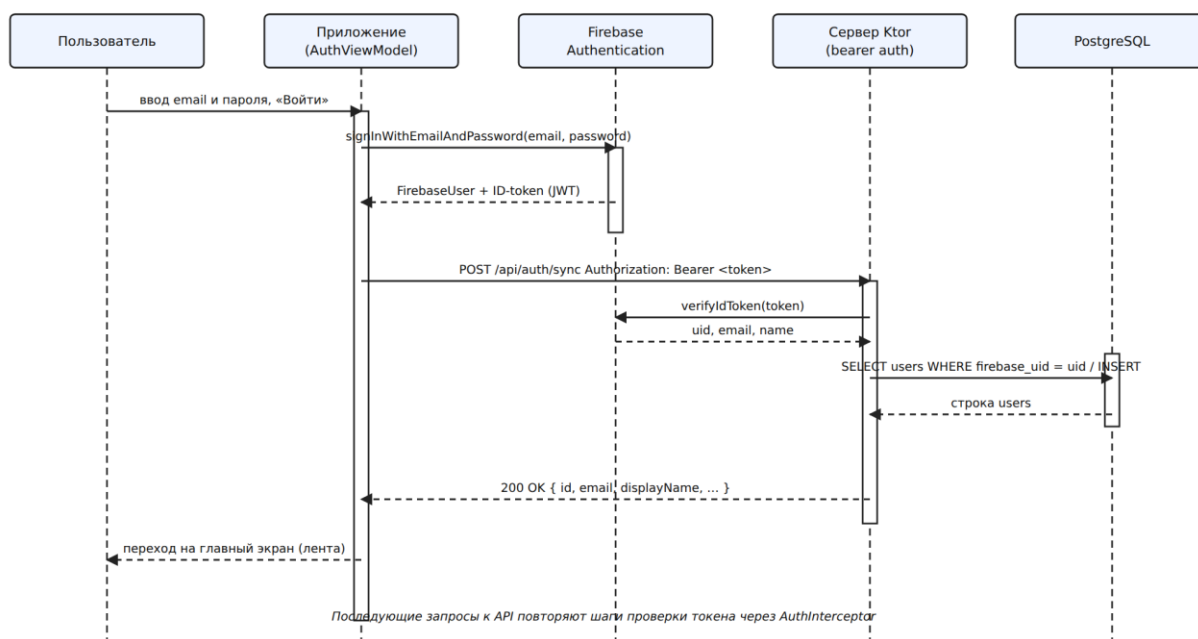


Рисунок 3.1 – Диаграмма последовательности входа в приложение

Для проверки API без клиента сервер поддерживает dev-режим, в котором заголовок `Authorization: Bearer dev:test1` принимается как токен пользователя `test1`. Это позволило отладить все конечные точки с помощью утилиты `curl` и `Postman` до подключения мобильного приложения.

3.3 Тестирование разработанного приложения

Тестирование проводилось методом ручного функционального тестирования по заранее составленным сценариям, а также проверкой API через утилиту `curl`. Стенд: сервер запущен на компьютере разработчика (JDK 17, PostgreSQL 16 в Docker), клиент — на эмуляторе Android (API 34) и физическом устройстве Android 13. Каждый сценарий содержит предусловие, шаги и ожидаемый результат; фактический результат сравнивался с ожидаемым. Перечень сценариев приведён в таблице 3.2, полные описания — в приложении Г.

Таблица 3.2 – Тестовые сценарии

№	Сценарий	Шаги	Ожидаемый результат	Итог
1	Регистрация нового пользователя	Открыть экран регистрации, ввести имя, email, пароль из 8 символов, нажать «Создать аккаунт»	Аккаунт создан, открыт экран ленты, в таблице <code>users</code> появилась запись	Пройден

№	Сценарий	Шаги	Ожидаемый результат	Итог
2	Регистрация с коротким паролем	Ввести пароль из 4 символов, нажать «Создать аккаунт»	Сообщение «Пароль должен быть не короче 6 символов», запрос не отправлен	Пройден
3	Вход с верными данными	Ввести email и пароль существующего пользователя, нажать «Войти»	Открыт экран ленты, выполнен запрос POST /api/auth/sync	Пройден
4	Вход с неверным паролем	Ввести неверный пароль, нажать «Войти»	Сообщение «Неверный email или пароль», пользователь остаётся на экране входа	Пройден
5	Просмотр ленты	Войти, открыть вкладку «Лента»	Отображены последние посты с автором, местом, оценкой и датой	Пройден
6	Создание места	На карте нажать «Добавить место», заполнить поля, выбрать категорию, нажать «Сохранить место»	Возврат на карту, новый маркер отображён, запись в таблице places	Пройден
7	Создание места без названия	Оставить поле «Название» пустым, нажать «Сохранить место»	Сообщение «Введите название места», запрос не отправлен	Пройден
8	Поиск мест по названию	Ввести «парк» в поисковую строку, нажать «Найти»	На карте остались только маркеры мест, содержащих «парк» в названии или описании	Пройден
9	Поиск мест рядом	Разрешить геолокацию, открыть карту	Карта центрирована на пользователя; загружены места в радиусе 10 км, отсортированные по расстоянию	Пройден
10	Публикация отзыва	Открыть место, ввести текст, выбрать оценку 5, нажать «Опубликовать»	Отзыв появился в начале списка, счётчик «Отзывы» увеличился на 1	Пройден
11	Отзыв с недопустимой оценкой (API)	Отправить POST /api/places/1/posts с rating = 7	Ответ 400 с сообщением «Оценка должна быть от 1 до 5»	Пройден
12	Запрос без токена (API)	Выполнить GET /api/feed без заголовка Authorization	Ответ 401 Unauthorized	Пройден
13	Удаление чужого места (API)	От пользователя test2 выполнить DELETE /api/places/{id} места пользователя test1	Ответ 403 с сообщением «Удалять место может только его автор»	Пройден
14	Редактирование профиля	В профиле нажать «Редактировать», изменить имя и описание, нажать «Сохранить»	Обновлённые данные отображены; PATCH /api/users/me вернул 200	Пройден
15	Отсутствие связи с сервером	Остановить сервер, обновить ленту	Сообщение «Нет связи с сервером. Проверьте, что сервер запущен.», приложение не завершилось аварийно	Пройден
16	Выход из аккаунта	В профиле нажать кнопку выхода	Открыт экран входа; повторный запуск приложения не восстанавливает сессию	Пройден

Все шестнадцать сценариев завершились успешно. В ходе тестирования были выявлены и устранены два дефекта: при повороте экрана на карте терялся введённый поисковый запрос (исправлено переносом текста запроса в состояние ViewModel) и при пустом ответе сервера лента показывала индикатор загрузки вместо заглушки (исправлено добавлением отдельной ветки состояния). Проверка требований НФТ-5 показала, что время ответа сервера на запрос ленты из 20 постов и поиск мест в радиусе 10 км при тестовой базе из 1 000 мест не превышает 150 мс.

Выводы по главе. Реализованы клиентская часть на Jetpack Compose с архитектурой Clean Architecture и серверная часть на Ktor с базой данных PostgreSQL. Интеграция выполнена через REST API с JSON и авторизацией по токенам Firebase, проверяемым на сервере. Функциональное тестирование по шестнадцати сценариям подтвердило выполнение всех функциональных требований ФТ-1 – ФТ-12 и корректную обработку ошибок.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработано клиент-серверное мобильное приложение социальной сети местоположений GeoSocial. Поставленная цель достигнута: пользователи могут регистрироваться, добавлять места на карту, публиковать отзывы с оценками, просматривать ленту активности и находить места рядом с собой.

В соответствии с задачами работы получены следующие результаты:

1. Описана предметная область геосоциальной сети и проведён сравнительный анализ четырёх аналогов (Swarm, Яндекс Карты, 2ГИС, Google Maps), показавший отсутствие решений, сочетающих свободное добавление мест, социальную ленту и открытость для разработчика.
2. Обоснован выбор инструментальных средств: Kotlin как единый язык, Ktor и Exposed для сервера, PostgreSQL для хранения данных, Jetpack Compose, Retrofit и Hilt для клиента, Firebase Authentication как провайдер идентификации.
3. Сформулированы двенадцать функциональных и девять нефункциональных требований, построена диаграмма вариантов использования.
4. Спроектированы макеты семи экранов и схема межэкранного взаимодействия с нижней панелью навигации.
5. Построены функциональная модель в нотации IDEF0, диаграммы классов клиентской и серверной частей, логическая модель базы данных из трёх таблиц.
6. Реализован сервер с пятнадцатью конечными точками REST API, проверкой токенов, двухэтапным геопространственным поиском и автоматическим созданием схемы базы данных.

7. Реализован Android-клиент по принципам Clean Architecture с однонаправленным потоком данных, картой Google Maps и определением местоположения.
8. Проведено функциональное тестирование по шестнадцати сценариям, подтвердившее выполнение всех требований.

Практическая значимость работы заключается в получении рабочего прототипа, который может быть использован как основа для локального городского сервиса, а также в отработке типового стека современной мобильной разработки. Полученные навыки — проектирование REST API, организация слоёв Clean Architecture, интеграция внешнего провайдера аутентификации с собственным сервером — применимы в промышленной разработке.

Возможные направления дальнейшего развития приложения:

- загрузка фотографий мест и отзывов с хранением файлов на сервере (поле `photo_url` уже зарезервировано в модели данных);
- социальные функции: лайки, комментарии к отзывам, подписки на пользователей и персональная лента;
- использование расширения PostGIS для индексированного геопространственного поиска при росте объёма данных;
- кэширование данных на клиенте средствами Room для работы в офлайн-режиме и пагинация ленты через библиотеку Paging 3;
- переход на HTTPS и публикация сервера в облаке для использования приложения вне локальной сети.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алпатов, А. Н. Архитектура, проектирование и разработка программных средств : учебное пособие / А. Н. Алпатов, И. Е. Рогов. — Москва : РТУ МИРЭА, 2023. — 120 с. — ISBN 978-5-7339-1972-0. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/386189> (дата обращения: 15.03.2026).
2. Swarm by Foursquare : [Электронный ресурс]. — URL: <https://www.swarmapp.com/> (дата обращения: 15.03.2026).
3. Яндекс Карты — отзывы и оценки организаций : [Электронный ресурс]. — URL: <https://yandex.ru/maps/> (дата обращения: 15.03.2026).
4. 2ГИС — карты и справочник организаций : [Электронный ресурс]. — URL: <https://2gis.ru/> (дата обращения: 15.03.2026).
5. Google Maps Local Guides : [Электронный ресурс]. — URL: <https://maps.google.com/localguides/> (дата обращения: 15.03.2026).
6. Сомон, П. И. Волшебство Kotlin : руководство / П. И. Сомон ; перевод с английского А. Н. Киселева. — Москва : ДМК Пресс, 2020. — 536 с. — ISBN 978-5-97060-801-2. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/140599> (дата обращения: 15.03.2026).
7. Ktor Documentation — Creating a server : [Электронный ресурс] / JetBrains. — URL: <https://ktor.io/docs/server-create-a-new-project.html> (дата обращения: 20.03.2026).
8. Exposed — Kotlin SQL Framework : документация : [Электронный ресурс] / JetBrains. — URL: <https://www.jetbrains.com/help/exposed/home.html> (дата обращения: 20.03.2026).
9. PostgreSQL 16 Documentation : [Электронный ресурс] / The PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 20.03.2026).

10. Firebase Authentication — Documentation : [Электронный ресурс] / Google. — URL: <https://firebase.google.com/docs/auth> (дата обращения: 22.03.2026).
11. Jetpack Compose — Android Developers : [Электронный ресурс] / Google. — URL: <https://developer.android.com/develop/ui/compose> (дата обращения: 22.03.2026).
12. Retrofit — A type-safe HTTP client for Android and Java : [Электронный ресурс] / Square. — URL: <https://square.github.io/retrofit/> (дата обращения: 22.03.2026).
13. Dependency injection with Hilt — Android Developers : [Электронный ресурс] / Google. — URL: <https://developer.android.com/training/dependency-injection/hilt-android> (дата обращения: 22.03.2026).
14. Maps Compose Library — Google Maps Platform : [Электронный ресурс] / Google. — URL: <https://developers.google.com/maps/documentation/android-sdk/maps-compose> (дата обращения: 22.03.2026).
15. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург : Питер, 2023. — 352 с. — (Библиотека программиста).
16. Р 50.1.028-2001. Информационные технологии поддержки жизненного цикла продукции. Методология функционального моделирования. — Москва : Госстандарт России, 2001. — 49 с.
17. Калгина, И. С. Разработка мобильных приложений : учебное пособие / И. С. Калгина. — Чита : ЗабГУ, 2022. — 163 с. — ISBN 978-5-9293-3137-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/363323> (дата обращения: 15.03.2026).
18. Guide to app architecture — Android Developers : [Электронный ресурс] / Google. — URL: <https://developer.android.com/topic/architecture> (дата обращения: 22.03.2026).
19. Kotlin Coroutines Guide : [Электронный ресурс] / JetBrains. — URL: <https://kotlinlang.org/docs/coroutines-guide.html> (дата обращения: 22.03.2026).

20. HikariCP — a solid, high-performance JDBC connection pool :
[Электронный ресурс]. — URL: <https://github.com/brettwooldridge/HikariCP> (дата обращения: 22.03.2026).

ПРИЛОЖЕНИЯ

Приложение А. Листинг серверной части приложения

Приложение Б. Листинг клиентской части приложения

Приложение В. Экраны мобильного приложения

Приложение Г. Тестовые сценарии

Приложение А. Листинг серверной части приложения

Файл build.gradle.kts

```
plugins {
    kotlin("jvm") version "2.0.21"
    kotlin("plugin.serialization") version "2.0.21"
    id("io.ktor.plugin") version "3.0.1"
}

group = "com.example.geosocial"
version = "1.0.0"

application {
    mainClass.set("io.ktor.server.netty.EngineMain")
}

repositories {
    mavenCentral()
}

val ktorVersion = "3.0.1"
val exposedVersion = "0.55.0"

dependencies {
    // Ktor server
    implementation("io.ktor:ktor-server-core-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-netty-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-content-negotiation-jvm:$ktorVersion")
    implementation("io.ktor:ktor-serialization-kotlinx-json-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-auth-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-status-pages-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-call-logging-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-cors-jvm:$ktorVersion")
    implementation("io.ktor:ktor-server-default-headers-jvm:$ktorVersion")

    // База данных
    implementation("org.jetbrains.exposed:exposed-core:$exposedVersion")
    implementation("org.jetbrains.exposed:exposed-jdbc:$exposedVersion")
    implementation("org.jetbrains.exposed:exposed-java-time:$exposedVersion")
    implementation("org.postgresql:postgresql:42.7.4")
    implementation("com.zaxxer:HikariCP:5.1.0")

    // Firebase Admin (проверка ID-токенов)
    implementation("com.google.firebase:firebase-admin:9.4.1")

    implementation("ch.qos.logback:logback-classic:1.5.8")

    testImplementation("io.ktor:ktor-server-test-host-jvm:$ktorVersion")
    testImplementation(kotlin("test"))
}

java {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}

kotlin {
    compilerOptions {
        jvmTarget.set(org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17)
    }
}
```

Файл docker-compose.yml

```
services:
    db:
        image: postgres:16-alpine
```

```

    container_name: geosocial-db
    environment:
      POSTGRES_DB: geosocial
      POSTGRES_USER: geosocial
      POSTGRES_PASSWORD: geosocial
    ports:
      - "5432:5432"
    volumes:
      - geosocial-pgdata:/var/lib/postgresql/data

volumes:
  geosocial-pgdata:

```

Файл settings.gradle.kts

```
rootProject.name = "geosocial-server"
```

Файл src/main/kotlin/com/example/geosocial/Application.kt

```

package com.example.geosocial

import com.example.geosocial.config.AppConfig
import com.example.geosocial.data.PlaceRepository
import com.example.geosocial.data.PostRepository
import com.example.geosocial.data.UserRepository
import com.example.geosocial.db.DatabaseFactory
import com.example.geosocial.plugins.configureMonitoring
import com.example.geosocial.plugins.configureSecurity
import com.example.geosocial.plugins.configureSerialization
import com.example.geosocial.routes.configureRouting
import com.example.geosocial.security.FirebaseAuthService
import io.ktor.server.application.*

fun main(args: Array<String>) = io.ktor.server.netty.EngineMain.main(args)

fun Application.module() {
    val config = AppConfig.from(this)

    DatabaseFactory.init(config)

    val userRepository = UserRepository()
    val placeRepository = PlaceRepository()
    val postRepository = PostRepository()
    val authService = FirebaseAuthService(config)

    configureSerialization()
    configureMonitoring()
    configureSecurity(authService, userRepository)
    configureRouting(userRepository, placeRepository, postRepository)

    log.info("GeoSocial API запущен")
}

```

Файл src/main/kotlin/com/example/geosocial/config/AppConfig.kt

```

package com.example.geosocial.config

import io.ktor.server.application.*

/** Настройки приложения, читаются из application.conf */
data class AppConfig(
    val dbUrl: String,
    val dbUser: String,
    val dbPassword: String,
    val firebaseServiceAccountPath: String,
    val authDevMode: Boolean
) {
    companion object {

```

```

fun from(application: Application): AppConfig {
    val cfg = application.environment.config
    return AppConfig(
        dbUrl = cfg.property("db.url").getString(),
        dbUser = cfg.property("db.user").getString(),
        dbPassword = cfg.property("db.password").getString(),
        firebaseServiceAccountPath =
            cfg.property("firebase.serviceAccountPath").getString(),
        authDevMode = cfg.property("firebase.devMode").getString().toBoolean()
    )
}
}
}

```

Файл `src/main/kotlin/com/example/geosocial/data/PlaceRepository.kt`

```

package com.example.geosocial.data

import com.example.geosocial.db.DatabaseFactory.dbQuery
import com.example.geosocial.db.Places
import com.example.geosocial.db.Posts
import com.example.geosocial.db.Users
import com.example.geosocial.domain.AppException
import com.example.geosocial.domain.GeoUtils
import com.example.geosocial.dto.CreatePlaceRequest
import com.example.geosocial.dto.PlaceDto
import org.jetbrains.exposed.sql.*
import org.jetbrains.exposed.sql.SqlExpressionBuilder.eq
import org.jetbrains.exposed.sql.SqlExpressionBuilder.greaterEq
import org.jetbrains.exposed.sql.SqlExpressionBuilder.lessEq
import org.jetbrains.exposed.sql.SqlExpressionBuilder.like
import org.jetbrains.exposed.sql.deleteWhere
import java.time.LocalDateTime

class PlaceRepository {

    private val joined = Places.innerJoin(Users, { Places.authorId }, { Users.id })

    /**
     * Поиск мест. Если переданы координаты — возвращаются места в радиусе
     * (сначала грубый отбор по прямоугольнику в SQL, затем точное расстояние).
     */
    suspend fun search(
        lat: Double?,
        lon: Double?,
        radiusKm: Double,
        query: String?,
        category: String?,
        limit: Int,
        offset: Int
    ): List<PlaceDto> = dbQuery {
        var condition: Op<Boolean> = Op.TRUE

        if (lat != null && lon != null) {
            val box = GeoUtils.boundingBox(lat, lon, radiusKm)
            condition = condition and
                (Places.latitude greaterEq box.minLat) and
                (Places.latitude lessEq box.maxLat) and
                (Places.longitude greaterEq box.minLon) and
                (Places.longitude lessEq box.maxLon)
        }
        if (!query.isNullOrEmpty()) {
            condition = condition and (
                (Places.title.lowerCase() like "%${query.lowercase()}%") or
                (Places.description.lowerCase() like
                    "%${query.lowercase()}%")
            )
        }
        if (!category.isNullOrEmpty()) {
            condition = condition and (Places.category eq category)
        }
    }
}

```

```

    }

    val rows = joined.selectAll().where { condition }
        .orderBy(Places.createdAt to SortOrder.DESC)
        .limit(if (lat != null) 500 else limit)
        .offset(offset.toLong())
        .toList()

    val counts = postsCountFor(rows.map { it[Places.id].value })
    var result = rows.map { toDto(it, counts, lat, lon) }

    if (lat != null && lon != null) {
        result = result
            .filter { (it.distanceKm ?: Double.MAX_VALUE) <= radiusKm }
            .sortedBy { it.distanceKm }
            .take(limit)
    }
    result
}

suspend fun findById(id: Long, lat: Double? = null, lon: Double? = null): PlaceDto =
dbQuery {
    val row = joined.selectAll().where { Places.id eq id }.singleOrNull()
        ?: throw AppException.NotFound("Место")
    toDto(row, postsCountFor(listOf(id)), lat, lon)
}

suspend fun findByAuthor(authorId: Long): List<PlaceDto> = dbQuery {
    val rows = joined.selectAll().where { Places.authorId eq authorId }
        .orderBy(Places.createdAt to SortOrder.DESC)
        .toList()
    val counts = postsCountFor(rows.map { it[Places.id].value })
    rows.map { toDto(it, counts, null, null) }
}

suspend fun create(authorId: Long, request: CreatePlaceRequest): PlaceDto {
    validate(request)
    val newId = dbQuery {
        Places.insertAndGetId {
            it[title] = request.title.trim()
            it[description] = request.description.trim()
            it[category] = request.category.trim()
            it[latitude] = request.latitude
            it[longitude] = request.longitude
            it[address] = request.address?.trim()
            it[Places.authorId] = authorId
            it[createdAt] = LocalDateTime.now()
        }.value
    }
    return findById(newId)
}

suspend fun delete(id: Long, requesterId: Long) = dbQuery {
    val row = Places.selectAll().where { Places.id eq id }.singleOrNull()
        ?: throw AppException.NotFound("Место")
    if (row[Places.authorId].value != requesterId) {
        throw AppException.Forbidden("Удалять место может только его автор")
    }
    Places.deleteWhere { Places.id eq id }
}

private fun validate(request: CreatePlaceRequest) {
    if (request.title.isBlank()) throw AppException.BadRequest("Название не может быть пустым")
    if (request.latitude !in -90.0..90.0) throw AppException.BadRequest("Некорректная широта")
    if (request.longitude !in -180.0..180.0) throw
AppException.BadRequest("Некорректная долгота")
}

```

```

private fun postsCountFor(placeIds: List<Long>): Map<Long, Long> {
    if (placeIds.isEmpty()) return emptyMap()
    val countCol = Posts.id.count()
    return Posts
        .select(Posts.placeId, countCol)
        .where { Posts.placeId inList placeIds }
        .groupBy(Posts.placeId)
        .associate { it[Posts.placeId].value to it[countCol] }
}

companion object {
    fun toDto(
        row: ResultRow,
        counts: Map<Long, Long> = emptyMap(),
        lat: Double? = null,
        lon: Double? = null
    ): PlaceDto {
        val id = row[Places.id].value
        val placeLat = row[Places.latitude]
        val placeLon = row[Places.longitude]
        return PlaceDto(
            id = id,
            title = row[Places.title],
            description = row[Places.description],
            category = row[Places.category],
            latitude = placeLat,
            longitude = placeLon,
            address = row[Places.address],
            author = UserRepository.toShortDto(row),
            postsCount = counts[id] ?: 0,
            distanceKm = if (lat != null && lon != null)
                GeoUtils.distanceKm(lat, lon, placeLat, placeLon) else null,
            createdAt = row[Places.createdAt].toString()
        )
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/data/PostRepository.kt`

```

package com.example.geosocial.data

import com.example.geosocial.db.DatabaseFactory.dbQuery
import com.example.geosocial.db.Places
import com.example.geosocial.db.Posts
import com.example.geosocial.db.Users
import com.example.geosocial.domain.AppException
import com.example.geosocial.dto.CreatePostRequest
import com.example.geosocial.dto.PostDto
import org.jetbrains.exposed.sql.*
import org.jetbrains.exposed.sql.deleteWhere
import java.time.LocalDateTime

class PostRepository {

    private val joined = Posts
        .innerJoin(Places, { Posts.placeId }, { Places.id })
        .innerJoin(Users, { Posts.authorId }, { Users.id })

    /** Общая лента: последние посты всех пользователей. */
    suspend fun feed(limit: Int, offset: Int): List<PostDto> = dbQuery {
        joined.selectAll()
            .orderBy(Posts.createdAt to SortOrder.DESC)
            .limit(limit)
            .offset(offset.toLong())
            .map(::toDto)
    }

    suspend fun byPlace(placeId: Long): List<PostDto> = dbQuery {
        joined.selectAll().where { Posts.placeId eq placeId }
    }
}

```

```

        .orderBy(Posts.createdAt to SortOrder.DESC)
        .map(::toDto)
    }

    suspend fun byAuthor(authorId: Long): List<PostDto> = dbQuery {
        joined.selectAll().where { Posts.authorId eq authorId }
        .orderBy(Posts.createdAt to SortOrder.DESC)
        .map(::toDto)
    }

    suspend fun create(placeId: Long, authorId: Long, request: CreatePostRequest):
    PostDto = dbQuery {
        if (request.text.isBlank()) throw AppException.BadRequest("Текст поста не может
        быть пустым")
        request.rating?.let {
            if (it !in 1..5) throw AppException.BadRequest("Оценка должна быть от 1 до
        5")
        }
        Places.selectAll().where { Places.id eq placeId }.singleOrNull()
        ?: throw AppException.NotFound("Место")

        val newId = Posts.insertAndGetId {
            it[Posts.placeId] = placeId
            it[Posts.authorId] = authorId
            it[text] = request.text.trim()
            it[photoUrl] = request.photoUrl
            it[rating] = request.rating
            it[createdAt] = LocalDateTime.now()
        }
        joined.selectAll().where { Posts.id eq newId }.single().let(::toDto)
    }

    suspend fun delete(postId: Long, requesterId: Long) = dbQuery {
        val row = Posts.selectAll().where { Posts.id eq postId }.singleOrNull()
        ?: throw AppException.NotFound("Пост")
        if (row[Posts.authorId].value != requesterId) {
            throw AppException.Forbidden("Удалять пост может только его автор")
        }
        Posts.deleteWhere { Posts.id eq postId }
    }

    private fun toDto(row: ResultRow) = PostDto(
        id = row[Posts.id].value,
        placeId = row[Posts.placeId].value,
        placeTitle = row[Places.title],
        text = row[Posts.text],
        photoUrl = row[Posts.photoUrl],
        rating = row[Posts.rating],
        author = UserRepository.toShortDto(row),
        createdAt = row[Posts.createdAt].toString()
    )
}

```

Файл `src/main/kotlin/com/example/geosocial/data/UserRepository.kt`

```

package com.example.geosocial.data

import com.example.geosocial.db.DatabaseFactory.dbQuery
import com.example.geosocial.db.Users
import com.example.geosocial.domain.AppException
import com.example.geosocial.dto.UpdateProfileRequest
import com.example.geosocial.dto.UserDto
import com.example.geosocial.dto.UserShortDto
import org.jetbrains.exposed.sql.ResultRow
import org.jetbrains.exposed.sql.insertAndGetId
import org.jetbrains.exposed.sql.selectAll
import org.jetbrains.exposed.sql.update
import java.time.LocalDateTime

```

```

class UserRepository {

    suspend fun findByFirebaseUid(uid: String): UserDto? = dbQuery {
        Users.selectAll().where { Users.firebaseUid eq uid }.singleOrNull()?.let(::toDto)
    }

    suspend fun findById(id: Long): UserDto? = dbQuery {
        Users.selectAll().where { Users.id eq id }.singleOrNull()?.let(::toDto)
    }

    /**
     * Создаёт профиль при первом входе или возвращает существующий.
     * Вызывается клиентом сразу после успешной авторизации в Firebase.
     */
    suspend fun syncUser(
        firebaseUid: String,
        email: String,
        displayName: String
    ): UserDto = dbQuery {
        val existing = Users.selectAll().where { Users.firebaseUid eq firebaseUid }.singleOrNull()
        if (existing != null) {
            toDto(existing)
        } else {
            val id = Users.insertAndGetId {
                it[Users.firebaseUid] = firebaseUid
                it[Users.email] = email
                it[Users.displayName] = displayName.ifBlank { email.substringBefore("@")
            }

            it[createdAt] = LocalDateTime.now()
        }
        Users.selectAll().where { Users.id eq id }.single().let(::toDto)
    }

    suspend fun updateProfile(userId: Long, request: UpdateProfileRequest): UserDto =
    dbQuery {
        val updated = Users.update({ Users.id eq userId }) { row ->
            request.displayName?.let { row[displayName] = it }
            request.bio?.let { row[bio] = it }
            request.avatarUrl?.let { row[avatarUrl] = it }
        }
        if (updated == 0) throw AppException.NotFound("Пользователь")
        Users.selectAll().where { Users.id eq userId }.single().let(::toDto)
    }

    companion object {
        fun toDto(row: ResultRow) = UserDto(
            id = row[Users.id].value,
            email = row[Users.email],
            displayName = row[Users.displayName],
            bio = row[Users.bio],
            avatarUrl = row[Users.avatarUrl],
            createdAt = row[Users.createdAt].toString()
        )

        fun toShortDto(row: ResultRow) = UserShortDto(
            id = row[Users.id].value,
            displayName = row[Users.displayName],
            avatarUrl = row[Users.avatarUrl]
        )
    }
}

```

Файл src/main/kotlin/com/example/geosocial/db/DatabaseFactory.kt

```

package com.example.geosocial.db

import com.example.geosocial.config.AppConfig

```

```

import com.zaxxer.hikari.HikariConfig
import com.zaxxer.hikari.HikariDataSource
import kotlinx.coroutines.Dispatchers
import org.jetbrains.exposed.sql.Database
import org.jetbrains.exposed.sql.SchemaUtils
import org.jetbrains.exposed.sql.Transaction
import org.jetbrains.exposed.sql.transactions.experimental.newSuspendedTransaction
import org.jetbrains.exposed.sql.transactions.transaction

object DatabaseFactory {

    fun init(config: AppConfig) {
        val hikariConfig = HikariConfig().apply {
            driverClassName = "org.postgresql.Driver"
            jdbcUrl = config.dbUrl
            username = config.dbUser
            password = config.dbPassword
            maximumPoolSize = 10
            isAutoCommit = false
            transactionIsolation = "TRANSACTION_REPEATABLE_READ"
            validate()
        }
        Database.connect(HikariDataSource(hikariConfig))

        // Для учебного проекта схему создаём автоматически
        transaction {
            SchemaUtils.createMissingTablesAndColumns(Users, Places, Posts)
        }
    }

    /** Выполнить запрос к БД в suspend-контексте. */
    suspend fun <T> dbQuery(block: suspend Transaction.() -> T): T =
        newSuspendedTransaction(Dispatchers.IO) { block() }
}

```

Файл `src/main/kotlin/com/example/geosocial/db/Tables.kt`

```

package com.example.geosocial.db

import org.jetbrains.exposed.dao.id.LongIdTable
import org.jetbrains.exposed.sql.ReferenceOption
import org.jetbrains.exposed.sql.javatime.datetime

/** Пользователи. firebaseUid — идентификатор из Firebase Authentication. */
object Users : LongIdTable("users") {
    val firebaseUid = varchar("firebase_uid", 128).uniqueIndex()
    val email = varchar("email", 255)
    val displayName = varchar("display_name", 100)
    val bio = varchar("bio", 500).nullable()
    val avatarUrl = varchar("avatar_url", 500).nullable()
    val createdAt = datetime("created_at")
}

/** Места на карте — основная сущность «социальной сети местоположений». */
object Places : LongIdTable("places") {
    val title = varchar("title", 150)
    val description = varchar("description", 1000)
    val category = varchar("category", 50)
    val latitude = double("latitude")
    val longitude = double("longitude")
    val address = varchar("address", 300).nullable()
    val authorId = reference("author_id", Users, onDelete = ReferenceOption.CASCADE)
    val createdAt = datetime("created_at")
}

/** Посты (отзывы/чек-ины) пользователей в конкретном месте. */
object Posts : LongIdTable("posts") {
    val placeId = reference("place_id", Places, onDelete = ReferenceOption.CASCADE)
    val authorId = reference("author_id", Users, onDelete = ReferenceOption.CASCADE)
}

```

```

        val text = varchar("text", 2000)
        val photoUrl = varchar("photo_url", 500).nullable()
        val rating = integer("rating").nullable()
        val createdAt = datetime("created_at")
    }

```

Файл `src/main/kotlin/com/example/geosocial/domain/Errors.kt`

```

package com.example.geosocial.domain

/** Ошибки уровня приложения, превращаются в HTTP-ответы в StatusPages. */
sealed class AppException(message: String) : RuntimeException(message) {
    class NotFound(what: String) : AppException("$what не найден")
    class BadRequest(message: String) : AppException(message)
    class Unauthorized(message: String = "Требуется авторизация") : AppException(message)
    class Forbidden(message: String = "Нет прав на это действие") : AppException(message)
}

```

Файл `src/main/kotlin/com/example/geosocial/domain/GeoUtils.kt`

```

package com.example.geosocial.domain

import kotlin.math.atan2
import kotlin.math.cos
import kotlin.math.pow
import kotlin.math.sin
import kotlin.math.sqrt

object GeoUtils {
    private const val EARTH_RADIUS_KM = 6371.0

    /** Расстояние между двумя точками по формуле гаверсинусов, км. */
    fun distanceKm(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Double {
        val dLat = Math.toRadians(lat2 - lat1)
        val dLon = Math.toRadians(lon2 - lon1)
        val a = sin(dLat / 2).pow(2) +
            cos(Math.toRadians(lat1)) * cos(Math.toRadians(lat2)) * sin(dLon /
2).pow(2)
        return EARTH_RADIUS_KM * 2 * atan2(sqrt(a), sqrt(1 - a))
    }

    /** Границы прямоугольника вокруг точки — чтобы отсечь лишнее прямо в SQL. */
    fun boundingBox(lat: Double, lon: Double, radiusKm: Double): BoundingBox {
        val latDelta = radiusKm / 111.0
        val lonDelta = radiusKm / (111.0 * cos(Math.toRadians(lat)).coerceAtLeast(0.01))
        return BoundingBox(lat - latDelta, lat + latDelta, lon - lonDelta, lon +
lonDelta)
    }

    data class BoundingBox(
        val minLat: Double,
        val maxLat: Double,
        val minLon: Double,
        val maxLon: Double
    )
}

```

Файл `src/main/kotlin/com/example/geosocial/dto/Dto.kt`

```

package com.example.geosocial.dto

import kotlinx.serialization.Serializable

// ----- Пользователь -----

@Serializable
data class UserDto(
    val id: Long,

```

```

        val email: String,
        val displayName: String,
        val bio: String? = null,
        val avatarUrl: String? = null,
        val createdAt: String
    )

    @Serializable
    data class SyncUserRequest(
        val displayName: String? = null
    )

    @Serializable
    data class UpdateProfileRequest(
        val displayName: String? = null,
        val bio: String? = null,
        val avatarUrl: String? = null
    )

    // ----- Места -----

    @Serializable
    data class PlaceDto(
        val id: Long,
        val title: String,
        val description: String,
        val category: String,
        val latitude: Double,
        val longitude: Double,
        val address: String? = null,
        val author: UserShortDto,
        val postsCount: Long = 0,
        val distanceKm: Double? = null,
        val createdAt: String
    )

    @Serializable
    data class UserShortDto(
        val id: Long,
        val displayName: String,
        val avatarUrl: String? = null
    )

    @Serializable
    data class CreatePlaceRequest(
        val title: String,
        val description: String,
        val category: String,
        val latitude: Double,
        val longitude: Double,
        val address: String? = null
    )

    // ----- Посты -----

    @Serializable
    data class PostDto(
        val id: Long,
        val placeId: Long,
        val placeTitle: String,
        val text: String,
        val photoUrl: String? = null,
        val rating: Int? = null,
        val author: UserShortDto,
        val createdAt: String
    )

    @Serializable
    data class CreatePostRequest(
        val text: String,

```

```

        val photoUrl: String? = null,
        val rating: Int? = null
    )

    // ----- Служебное -----

    @Serializable
    data class ErrorResponse(val error: String, val message: String)

    @Serializable
    data class MessageResponse(val message: String)

```

Файл `src/main/kotlin/com/example/geosocial/plugins/Monitoring.kt`

```

package com.example.geosocial.plugins

import com.example.geosocial.domain.AppException
import com.example.geosocial.dto.ErrorResponse
import io.ktor.http.*
import io.ktor.server.application.*
import io.ktor.server.plugins.calllogging.*
import io.ktor.server.plugins.cors.routing.*
import io.ktor.server.plugins.defaultheaders.*
import io.ktor.server.plugins.statuspages.*
import io.ktor.server.response.*
import org.slf4j.event.Level

fun Application.configureMonitoring() {
    install(CallLogging) { level = Level.INFO }
    install(DefaultHeaders)
    install(CORS) {
        anyHost() // для учебного проекта
        allowHeader(HttpHeaders.Authorization)
        allowHeader(HttpHeaders.ContentType)
        allowMethod(HttpMethod.Get)
        allowMethod(HttpMethod.Post)
        allowMethod(HttpMethod.Patch)
        allowMethod(HttpMethod.Delete)
    }
    install(StatusPages) {
        exception<AppException> { call, cause ->
            val (status, code) = when (cause) {
                is AppException.NotFound -> HttpStatusCode.NotFound to "not_found"
                is AppException.BadRequest -> HttpStatusCode.BadRequest to "bad_request"
                is AppException.Unauthorized -> HttpStatusCode.Unauthorized to
"unauthorized"
                is AppException.Forbidden -> HttpStatusCode.Forbidden to "forbidden"
            }
            call.respond(status, ErrorResponse(code, cause.message ?: "Ошибка"))
        }
        exception<Throwable> { call, cause ->
            call.application.log.error("Необработанная ошибка", cause)
            call.respond(
                HttpStatusCode.InternalServerError,
                ErrorResponse("internal_error", cause.message ?: "Внутренняя ошибка сервера")
            )
        }
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/plugins/Security.kt`

```

package com.example.geosocial.plugins

import com.example.geosocial.data.UserRepository
import com.example.geosocial.security.FirebaseAuthService
import com.example.geosocial.security.UserPrincipal
import io.ktor.server.application.*

```

```

import io.ktor.server.auth.*

const val AUTH_FIREBASE = "firebase"

fun Application.configureSecurity(
    authService: FirebaseAuthService,
    userRepository: UserRepository
) {
    install(Authentication) {
        bearer(AUTH_FIREBASE) {
            realm = "geosocial"
            authenticate { tokenCredential ->
                val firebaseUser = authService.verify(tokenCredential.token) ?:
return@authenticate null
                // Профиль создаётся автоматически при первом запросе
                val user = userRepository.syncUser(
                    firebaseUid = firebaseUser.uid,
                    email = firebaseUser.email,
                    displayName = firebaseUser.displayName
                )
                UserPrincipal(
                    userId = user.id,
                    firebaseUid = firebaseUser.uid,
                    email = user.email,
                    displayName = user.displayName
                )
            }
        }
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/plugins/Serialization.kt`

```

package com.example.geosocial.plugins

import io.ktor.serialization.kotlinx.json.*
import io.ktor.server.application.*
import io.ktor.server.plugins.contentnegotiation.*
import kotlinx.serialization.json.Json

fun Application.configureSerialization() {
    install(ContentNegotiation) {
        json(Json {
            prettyPrint = true
            ignoreUnknownKeys = true
            encodeDefaults = true
        })
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/routes/PlaceRoutes.kt`

```

package com.example.geosocial.routes

import com.example.geosocial.data.PlaceRepository
import com.example.geosocial.data.PostRepository
import com.example.geosocial.domain.AppException
import com.example.geosocial.dto.CreatePlaceRequest
import com.example.geosocial.dto.CreatePostRequest
import com.example.geosocial.dto.MessageResponse
import com.example.geosocial.plugins.AUTH_FIREBASE
import com.example.geosocial.security.requireUser
import io.ktor.http.*
import io.ktor.server.auth.*
import io.ktor.server.request.*
import io.ktor.server.response.*
import io.ktor.server.routing.*

fun Route.placeRoutes(

```

```

        placeRepository: PlaceRepository,
        postRepository: PostRepository
    ) {
        authenticate(AUTH_FIREBASE) {
            route("/places") {

                /**
                 * GET /api/places?lat=55.75&lon=37.61&radius=5&query=кофе&category=cafe
                 * Без координат – просто последние добавленные места.
                 */
                get {
                    val lat = call.request.queryParameters["lat"]?.toDoubleOrNull()
                    val lon = call.request.queryParameters["lon"]?.toDoubleOrNull()
                    val radius = call.request.queryParameters["radius"]?.toDoubleOrNull() ?:
10.0

                    val query = call.request.queryParameters["query"]
                    val category = call.request.queryParameters["category"]
                    val limit =
call.request.queryParameters["limit"]?.toIntOrNull()?.coerceIn(1, 100) ?: 50
                    val offset = call.request.queryParameters["offset"]?.toIntOrNull() ?: 0

                    call.respond(
                        placeRepository.search(lat, lon, radius, query, category, limit,
offset)
                    )
                }

                get("/{id}") {
                    val id = call.longParam("id")
                    val lat = call.request.queryParameters["lat"]?.toDoubleOrNull()
                    val lon = call.request.queryParameters["lon"]?.toDoubleOrNull()
                    call.respond(placeRepository.findById(id, lat, lon))
                }

                post {
                    val user = call.requireUser()
                    val body = call.receive<CreatePlaceRequest>()
                    call.respond(HttpStatusCode.Created, placeRepository.create(user.userId,
body))
                }

                delete("/{id}") {
                    val user = call.requireUser()
                    placeRepository.delete(call.longParam("id"), user.userId)
                    call.respond(MessageResponse("Место удалено"))
                }

                /** Посты конкретного места. */
                get("/{id}/posts") {
                    call.respond(postRepository.byPlace(call.longParam("id")))
                }

                post("/{id}/posts") {
                    val user = call.requireUser()
                    val body = call.receive<CreatePostRequest>()
                    call.respond(
                        HttpStatusCode.Created,
                        postRepository.create(call.longParam("id"), user.userId, body)
                    )
                }
            }
        }
    }

    internal fun io.ktor.server.application.ApplicationCall.longParam(name: String): Long =
        parameters[name]?.toLongOrNull() ?: throw AppException.BadRequest("Некорректный
параметр $name")

```

Файл `src/main/kotlin/com/example/geosocial/routes/PostRoutes.kt`

```
package com.example.geosocial.routes

import com.example.geosocial.data.PostRepository
import com.example.geosocial.dto.MessageResponse
import com.example.geosocial.plugins.AUTH_FIREBASE
import com.example.geosocial.security.requireUser
import io.ktor.server.auth.*
import io.ktor.server.response.*
import io.ktor.server.routing.*

fun Route.postRoutes(postRepository: PostRepository) {
    authenticate(AUTH_FIREBASE) {

        /** Общая лента постов. */
        get("/feed") {
            val limit = call.request.queryParameters["limit"]?.toIntOrNull()?.coerceIn(1,
100) ?: 20
            val offset = call.request.queryParameters["offset"]?.toIntOrNull() ?: 0
            call.respond(postRepository.feed(limit, offset))
        }

        delete("/posts/{id}") {
            val user = call.requireUser()
            postRepository.delete(call.longParam("id"), user.userId)
            call.respond(MessageResponse("Пост удалён"))
        }
    }
}
```

Файл `src/main/kotlin/com/example/geosocial/routes/Routing.kt`

```
package com.example.geosocial.routes

import com.example.geosocial.data.PlaceRepository
import com.example.geosocial.data.PostRepository
import com.example.geosocial.data.UserRepository
import com.example.geosocial.dto.MessageResponse
import io.ktor.server.application.*
import io.ktor.server.response.*
import io.ktor.server.routing.*

fun Application.configureRouting(
    userRepository: UserRepository,
    placeRepository: PlaceRepository,
    postRepository: PostRepository
) {
    routing {
        get("/") {
            call.respond(MessageResponse("GeoSocial API работает. Смотри /api/health"))
        }
        route("/api") {
            get("/health") { call.respond(MessageResponse("ok")) }
            userRoutes(userRepository, placeRepository, postRepository)
            placeRoutes(placeRepository, postRepository)
            postRoutes(postRepository)
        }
    }
}
```

Файл `src/main/kotlin/com/example/geosocial/routes/UserRoutes.kt`

```
package com.example.geosocial.routes

import com.example.geosocial.data.PlaceRepository
import com.example.geosocial.data.PostRepository
import com.example.geosocial.data.UserRepository
import com.example.geosocial.domain.AppException
```

```

import com.example.geosocial.dto.UpdateProfileRequest
import com.example.geosocial.plugins.AUTH_FIREBASE
import com.example.geosocial.security.requireUser
import io.ktor.server.auth.*
import io.ktor.server.request.*
import io.ktor.server.response.*
import io.ktor.server.routing.*

fun Route.userRoutes(
    userRepository: UserRepository,
    placeRepository: PlaceRepository,
    postRepository: PostRepository
) {
    authenticate(AUTH_FIREBASE) {

        /** Регистрация/вход: профиль создаётся по Firebase-токену при первом обращении. */
        post("/auth/sync") {
            val user = call.requireUser()
            call.respond(
                userRepository.findById(user.userId) ?: throw
                AppException.NotFound("Пользователь")
            )
        }

        get("/users/me") {
            val user = call.requireUser()
            call.respond(
                userRepository.findById(user.userId) ?: throw
                AppException.NotFound("Пользователь")
            )
        }

        patch("/users/me") {
            val user = call.requireUser()
            val body = call.receive<UpdateProfileRequest>()
            call.respond(userRepository.updateProfile(user.userId, body))
        }

        get("/users/{id}") {
            val id = call.parameters["id"]?.toLongOrNull()
            ?: throw AppException.BadRequest("Некорректный id")
            call.respond(userRepository.findById(id) ?: throw
                AppException.NotFound("Пользователь"))
        }

        /** Места, созданные пользователем. */
        get("/users/{id}/places") {
            val id = call.parameters["id"]?.toLongOrNull()
            ?: throw AppException.BadRequest("Некорректный id")
            call.respond(placeRepository.findByIdAuthor(id))
        }

        /** Посты пользователя. */
        get("/users/{id}/posts") {
            val id = call.parameters["id"]?.toLongOrNull()
            ?: throw AppException.BadRequest("Некорректный id")
            call.respond(postRepository.byAuthor(id))
        }
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/security/FirebaseAuthService.kt`

```

package com.example.geosocial.security

import com.example.geosocial.config.AppConfig
import com.google.auth.oauth2.GoogleCredentials
import com.google.firebase.FirebaseApp

```

```

import com.google.firebase.FirebaseOptions
import com.google.firebase.auth.FirebaseAuth
import org.slf4j.LoggerFactory
import java.io.File
import java.io.FileInputStream

/** Данные пользователя, извлечённые из ID-токена Firebase. */
data class FirebaseUser(
    val uid: String,
    val email: String,
    val displayName: String
)

/**
 * Проверка Firebase ID-токена.
 *
 * Если путь к сервис-аккаунту не задан, а devMode = true, сервер принимает
 * «токены» вида `dev:<uid>` — это позволяет проверять API из Postman/curl
 * без настроенного Firebase.
 */
class FirebaseAuthService(private val config: AppConfig) {

    private val log = LoggerFactory.getLogger(FirebaseAuthService::class.java)
    private var initialized = false

    init {
        val path = config.firebaseServiceAccountPath
        if (path.isNotBlank() && File(path).exists()) {
            if (FirebaseApp.getApps().isEmpty()) {
                val options = FirebaseOptions.builder()
                    .setCredentials(GoogleCredentials.fromStream(FileInputStream(path)))
                    .build()
                FirebaseApp.initializeApp(options)
            }
            initialized = true
            log.info("Firebase Admin SDK инициализирован")
        } else {
            log.warn(
                "Firebase не настроен (firebase.serviceAccountPath пуст). " +
                "Dev-режим: ${config.authDevMode}"
            )
        }
    }

    fun verify(token: String): FirebaseUser? {
        if (config.authDevMode && token.startsWith("dev:")) {
            val uid = token.removePrefix("dev:").trim()
            if (uid.isEmpty()) return null
            return FirebaseUser(uid = uid, email = "$uid@dev.local", displayName = uid)
        }
        if (!initialized) return null
        return runCatching {
            val decoded = FirebaseAuth.getInstance().verifyIdToken(token)
            FirebaseUser(
                uid = decoded.uid,
                email = decoded.email ?: "${decoded.uid}@unknown.local",
                displayName = decoded.name ?: decoded.email?.substringBefore("@") ?:
                decoded.uid
            )
        }.onFailure { log.debug("Не удалось проверить токен: ${it.message}") }
        .getOrNull()
    }
}

```

Файл `src/main/kotlin/com/example/geosocial/security/UserPrincipal.kt`

```

package com.example.geosocial.security

import com.example.geosocial.domain.AppException
import io.ktor.server.application.*

```

```
import io.ktor.server.auth.*

/** Авторизованный пользователь текущего запроса. */
data class UserPrincipal(
    val userId: Long,
    val firebaseUid: String,
    val email: String,
    val displayName: String
)

fun ApplicationCall.requireUser(): UserPrincipal =
    principal<UserPrincipal>() ?: throw AppException.Unauthorized()
```

Файл `src/main/resources/application.conf`

```
ktor {
    deployment {
        port = 8080
        port = ${?PORT}
    }
    application {
        modules = [ com.example.geosocial.ApplicationKt.module ]
    }
}

db {
    url = "jdbc:postgresql://localhost:5432/geosocial"
    url = ${?DB_URL}
    user = "geosocial"
    user = ${?DB_USER}
    password = "geosocial"
    password = ${?DB_PASSWORD}
}

firebase {
    # Путь к JSON сервис-аккаунта Firebase. Если пусто — работает dev-режим.
    serviceAccountPath = ""
    serviceAccountPath = ${?FIREBASE_SERVICE_ACCOUNT}
    # В dev-режиме принимается токен вида "dev:<uid>" — удобно тестировать без Firebase
    devMode = true
    devMode = ${?AUTH_DEV_MODE}
}
```

Файл `src/main/resources/db/schema.sql`

```
-- Справочная схема БД (сервер создаёт таблицы автоматически при старте)

CREATE TABLE IF NOT EXISTS users (
    id                BIGSERIAL PRIMARY KEY,
    firebase_uid      VARCHAR(128) NOT NULL UNIQUE,
    email             VARCHAR(255) NOT NULL,
    display_name      VARCHAR(100) NOT NULL,
    bio               VARCHAR(500),
    avatar_url        VARCHAR(500),
    created_at        TIMESTAMP NOT NULL
);

CREATE TABLE IF NOT EXISTS places (
    id                BIGSERIAL PRIMARY KEY,
    title             VARCHAR(150) NOT NULL,
    description        VARCHAR(1000) NOT NULL,
    category          VARCHAR(50) NOT NULL,
    latitude           DOUBLE PRECISION NOT NULL,
    longitude          DOUBLE PRECISION NOT NULL,
    address            VARCHAR(300),
    author_id         BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    created_at        TIMESTAMP NOT NULL
);
```

```

CREATE TABLE IF NOT EXISTS posts (
  id          BIGSERIAL PRIMARY KEY,
  place_id    BIGINT NOT NULL REFERENCES places (id) ON DELETE CASCADE,
  author_id   BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  text        VARCHAR(2000) NOT NULL,
  photo_url   VARCHAR(500),
  rating      INT,
  created_at  TIMESTAMP NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_places_coords ON places (latitude, longitude);
CREATE INDEX IF NOT EXISTS idx_posts_place ON posts (place_id);

```

Приложение Б. Листинг клиентской части приложения

Файл app/build.gradle.kts

```
import java.util.Properties

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("org.jetbrains.kotlin.plugin.serialization")
    id("com.google.devtools.ksp")
    id("com.google.dagger.hilt.android")
    id("com.google.gms.google-services")
}

// Ключ Google Maps берём из local.properties, чтобы не хранить его в репозитории
val localProperties = Properties().apply {
    val file = rootProject.file("local.properties")
    if (file.exists()) file.inputStream().use { load(it) }
}
val mapsApiKey: String = localProperties.getProperty("MAPS_API_KEY") ?: ""

android {
    namespace = "com.example.geosocial"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.geosocial"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"

        manifestPlaceholders["MAPS_API_KEY"] = mapsApiKey

        // 10.0.2.2 – адрес хост-машины из эмулятора Android
        buildConfigField("String", "BASE_URL", "\"http://10.0.2.2:8080/\"")
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro")
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = "17"
    }
    buildFeatures {
        compose = true
        buildConfig = true
    }
    packaging {
        resources.excludes += "/META-INF/{AL2.0,LGPL2.1}"
    }
}

dependencies {
    // Compose
    val composeBom = platform("androidx.compose:compose-bom:2024.10.01")
    implementation(composeBom)
    implementation("androidx.compose.ui:ui")
}
```

```

implementation("androidx.compose.ui:ui-graphics")
implementation("androidx.compose.ui:ui-tooling-preview")
implementation("androidx.compose.material3:material3")
implementation("androidx.compose.material:material-icons-extended")
debugImplementation("androidx.compose.ui:ui-tooling")

implementation("androidx.core:core-ktx:1.13.1")
implementation("androidx.activity:activity-compose:1.9.3")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
implementation("androidx.navigation:navigation-compose:2.8.4")

// DI
implementation("com.google.dagger:hilt-android:2.52")
ksp("com.google.dagger:hilt-android-compiler:2.52")
implementation("androidx.hilt:hilt-navigation-compose:1.2.0")

// Сеть
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.okhttp3:okhttp:4.12.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-
converter:1.0.0")

// Firebase Authentication (email + пароль)
implementation(platform("com.google.firebase:firebase-bom:33.5.1"))
implementation("com.google.firebase:firebase-auth-ktx")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")

// Карта
implementation("com.google.maps.android:maps-compose:6.1.2")
implementation("com.google.android.gms:play-services-maps:19.0.0")
implementation("com.google.android.gms:play-services-location:21.3.0")

// Картинки
implementation("io.coil-kt:coil-compose:2.7.0")
}

```

Файл app/src/main/AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <application
        android:name=".GeoSocialApp"
        android:allowBackup="true"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:usesCleartextTraffic="true"
        android:theme="@style/Theme.GeoSocial">

        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="${MAPS_API_KEY}" />

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.GeoSocial">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

```

```

    </application>
</manifest>

```

Файл app/src/main/java/com/example/geosocial/GeoSocialApp.kt

```

package com.example.geosocial

import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class GeoSocialApp : Application()

```

Файл app/src/main/java/com/example/geosocial/MainActivity.kt

```

package com.example.geosocial

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.geosocial.presentation.navigation.AppNavigation
import com.example.geosocial.ui.theme.GeoSocialTheme
import dagger.hilt.android.AndroidEntryPoint

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            GeoSocialTheme {
                AppNavigation()
            }
        }
    }
}

```

Файл app/src/main/java/com/example/geosocial/data/auth/FirebaseAuthDataSource.kt

```

package com.example.geosocial.data.auth

import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.auth.UserProfileChangeRequest
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await
import javax.inject.Inject
import javax.inject.Singleton

/** Обёртка над Firebase Authentication (email + пароль). */
@Singleton
class FirebaseAuthDataSource @Inject constructor(
    private val auth: FirebaseAuth
) {

    val isLoggedIn: Flow<Boolean> = callbackFlow {
        val listener = FirebaseAuth.AuthStateListener { trySend(it.currentUser != null) }
        auth.addAuthStateListener(listener)
        awaitClose { auth.removeAuthStateListener(listener) }
    }

    fun currentUserId(): String? = auth.currentUser?.uid

    suspend fun signIn(email: String, password: String) {
        auth.signInWithEmailAndPassword(email, password).await()
    }
}

```

```

suspend fun signUp(email: String, password: String, displayName: String) {
    val result = auth.createUserWithEmailAndPassword(email, password).await()
    if (displayName.isNotBlank()) {
        result.user?.updateProfile(
            UserProfileChangeRequest.Builder().setDisplayName(displayName).build()
        )?.await()
    }
}

fun signOut() = auth.signOut()
}

```

Файл app/src/main/java/com/example/geosocial/data/mapper/Mappers.kt

```

package com.example.geosocial.data.mapper

import com.example.geosocial.data.remote.dto.PlaceDto
import com.example.geosocial.data.remote.dto.PostDto
import com.example.geosocial.data.remote.dto.UserDto
import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.model.User

fun UserDto.toDomain() = User(
    id = id,
    email = email,
    displayName = displayName,
    bio = bio,
    avatarUrl = avatarUrl
)

fun PlaceDto.toDomain() = Place(
    id = id,
    title = title,
    description = description,
    category = category,
    latitude = latitude,
    longitude = longitude,
    address = address,
    authorId = author.id,
    authorName = author.displayName,
    postsCount = postsCount,
    distanceKm = distanceKm,
    createdAt = createdAt
)

fun PostDto.toDomain() = Post(
    id = id,
    placeId = placeId,
    placeTitle = placeTitle,
    text = text,
    photoUrl = photoUrl,
    rating = rating,
    authorId = author.id,
    authorName = author.displayName,
    authorAvatar = author.avatarUrl,
    createdAt = createdAt
)

```

Файл app/src/main/java/com/example/geosocial/data/remote/AuthInterceptor.kt

```

package com.example.geosocial.data.remote

import com.google.firebase.auth.FirebaseAuth
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.tasks.await
import okhttp3.Interceptor
import okhttp3.Response

```

```

import javax.inject.Inject
import javax.inject.Singleton

/**
 * Подставляет в каждый запрос Firebase ID-token.
 * Сервер проверяет его через Firebase Admin SDK.
 */
@Singleton
class AuthInterceptor @Inject constructor(
    private val firebaseAuth: FirebaseAuth
) : Interceptor {

    override fun intercept(chain: Interceptor.Chain): Response {
        val token = runBlocking {
            firebaseAuth.currentUser?.let { user ->
                runCatching { user.getIdToken(false).await().token }.getOrNull()
            }
        }
        val request = chain.request().newBuilder()
            .apply { token?.let { header("Authorization", "Bearer $it") } }
            .build()
        return chain.proceed(request)
    }
}

```

Файл app/src/main/java/com/example/geosocial/data/remote/GeoSocialApi.kt

```

package com.example.geosocial.data.remote

import com.example.geosocial.data.remote.dto.*
import retrofit2.http.*

/** Описание REST API сервера. */
interface GeoSocialApi {

    @POST("api/auth/sync")
    suspend fun syncUser(): UserDto

    @GET("api/users/me")
    suspend fun getMe(): UserDto

    @PATCH("api/users/me")
    suspend fun updateProfile(@Body body: UpdateProfileRequest): UserDto

    @GET("api/users/{id}/places")
    suspend fun getUserPlaces(@Path("id") userId: Long): List<PlaceDto>

    @GET("api/places")
    suspend fun getPlaces(
        @Query("lat") lat: Double? = null,
        @Query("lon") lon: Double? = null,
        @Query("radius") radius: Double? = null,
        @Query("query") query: String? = null,
        @Query("category") category: String? = null
    ): List<PlaceDto>

    @GET("api/places/{id}")
    suspend fun getPlace(@Path("id") id: Long): PlaceDto

    @POST("api/places")
    suspend fun createPlace(@Body body: CreatePlaceRequest): PlaceDto

    @DELETE("api/places/{id}")
    suspend fun deletePlace(@Path("id") id: Long): MessageResponse

    @GET("api/places/{id}/posts")
    suspend fun getPlacePosts(@Path("id") placeId: Long): List<PostDto>

    @POST("api/places/{id}/posts")

```

```

suspend fun addPost(@Path("id") placeId: Long, @Body body: CreatePostRequest):
PostDto

@GET("api/feed")
suspend fun getFeed(
    @Query("limit") limit: Int = 20,
    @Query("offset") offset: Int = 0
): List<PostDto>
}

```

Файл app/src/main/java/com/example/geosocial/data/remote/dto/Dtos.kt

```
package com.example.geosocial.data.remote.dto
```

```
import kotlinx.serialization.Serializable
```

```

@Serializable
data class UserDto(
    val id: Long,
    val email: String,
    val displayName: String,
    val bio: String? = null,
    val avatarUrl: String? = null,
    val createdAt: String = ""
)

```

```

@Serializable
data class UserShortDto(
    val id: Long,
    val displayName: String,
    val avatarUrl: String? = null
)

```

```

@Serializable
data class PlaceDto(
    val id: Long,
    val title: String,
    val description: String,
    val category: String,
    val latitude: Double,
    val longitude: Double,
    val address: String? = null,
    val author: UserShortDto,
    val postsCount: Long = 0,
    val distanceKm: Double? = null,
    val createdAt: String = ""
)

```

```

@Serializable
data class PostDto(
    val id: Long,
    val placeId: Long,
    val placeTitle: String,
    val text: String,
    val photoUrl: String? = null,
    val rating: Int? = null,
    val author: UserShortDto,
    val createdAt: String = ""
)

```

```

@Serializable
data class CreatePlaceRequest(
    val title: String,
    val description: String,
    val category: String,
    val latitude: Double,
    val longitude: Double,
    val address: String? = null
)

```

```

@Serializable
data class CreatePostRequest(
    val text: String,
    val photoUrl: String? = null,
    val rating: Int? = null
)

@Serializable
data class UpdateProfileRequest(
    val displayName: String? = null,
    val bio: String? = null,
    val avatarUrl: String? = null
)

@Serializable
data class MessageResponse(val message: String = "")

@Serializable
data class ErrorResponse(val error: String = "", val message: String = "")

```

Файл app/src/main/java/com/example/geosocial/data/repository/ApiCall.kt

```

package com.example.geosocial.data.repository

import retrofit2.HttpException
import java.io.IOException

/** Единая обработка сетевых ошибок для всех репозиторий. */
suspend fun <T> safeApiCall(block: suspend () -> T): Result<T> = try {
    Result.success(block())
} catch (e: HttpException) {
    val message = when (e.code()) {
        401 -> "Требуется вход в приложение"
        403 -> "Нет прав на это действие"
        404 -> "Не найдено"
        else -> "Ошибка сервера (${e.code()})"
    }
    Result.failure(Exception(message))
} catch (e: IOException) {
    Result.failure(Exception("Нет связи с сервером. Проверьте, что сервер запущен."))
} catch (e: Exception) {
    Result.failure(Exception(e.message ?: "Неизвестная ошибка"))
}

```

Файл app/src/main/java/com/example/geosocial/data/repository/AuthRepositoryImpl.kt

```

package com.example.geosocial.data.repository

import com.example.geosocial.data.auth.FirebaseAuthDataSource
import com.example.geosocial.data.remote.GeoSocialApi
import com.example.geosocial.domain.repository.AuthRepository
import com.google.firebase.auth.FirebaseAuthInvalidCredentialsException
import com.google.firebase.auth.FirebaseAuthInvalidUserException
import com.google.firebase.auth.FirebaseAuthUserCollisionException
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject

class AuthRepositoryImpl @Inject constructor(
    private val firebase: FirebaseAuthDataSource,
    private val api: GeoSocialApi
) : AuthRepository {

    override val isLoggedIn: Flow<Boolean> = firebase.isLoggedIn

    override suspend fun signIn(email: String, password: String): Result<Unit> = try {
        firebase.signIn(email, password)
        // создаём/подтягиваем профиль на нашем сервере
    }

```

```

        runCatching { api.syncUser() }
        Result.success(Unit)
    } catch (e: Exception) {
        Result.failure(Exception(mapFirebaseError(e)))
    }

    override suspend fun signUp(
        email: String,
        password: String,
        displayName: String
    ): Result<Unit> = try {
        firebase.signUp(email, password, displayName)
        runCatching { api.syncUser() }
        Result.success(Unit)
    } catch (e: Exception) {
        Result.failure(Exception(mapFirebaseError(e)))
    }

    override fun signOut() = firebase.signOut()

    private fun mapFirebaseError(e: Exception): String = when (e) {
        is FirebaseAuthInvalidUserException -> "Пользователь с таким email не найден"
        is FirebaseAuthInvalidCredentialsException -> "Неверный email или пароль"
        is FirebaseAuthUserCollisionException -> "Такой email уже зарегистрирован"
        else -> e.message ?: "Ошибка авторизации"
    }
}

```

Файл app/src/main/java/com/example/geosocial/data/repository/PlaceRepositoryImpl.kt

```

package com.example.geosocial.data.repository

import com.example.geosocial.data.mapper.toDomain
import com.example.geosocial.data.remote.GeoSocialApi
import com.example.geosocial.data.remote.dto.CreatePlaceRequest
import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.repository.PlaceRepository
import javax.inject.Inject

class PlaceRepositoryImpl @Inject constructor(
    private val api: GeoSocialApi
) : PlaceRepository {

    override suspend fun getPlaces(
        latitude: Double?,
        longitude: Double?,
        radiusKm: Double,
        query: String?,
        category: String?
    ): Result<List<Place>> = safeApiCall {
        api.getPlaces(
            lat = latitude,
            lon = longitude,
            radius = if (latitude != null) radiusKm else null,
            query = query?.takeIf { it.isNotBlank() },
            category = category
        ).map { it.toDomain() }
    }

    override suspend fun getPlace(id: Long): Result<Place> =
        safeApiCall { api.getPlace(id).toDomain() }

    override suspend fun createPlace(
        title: String,
        description: String,
        category: String,
        latitude: Double,
        longitude: Double,
        address: String?
    ) {

```

```

    ): Result<Place> = safeApiCall {
        api.createPlace(
            CreatePlaceRequest(title, description, category, latitude, longitude,
address)
        ).toDomain()
    }

    override suspend fun deletePlace(id: Long): Result<Unit> = safeApiCall {
        api.deletePlace(id)
        Unit
    }
}

```

Файл app/src/main/java/com/example/geosocial/data/repository/PostRepositoryImpl.kt

```

package com.example.geosocial.data.repository

import com.example.geosocial.data.mapper.toDomain
import com.example.geosocial.data.remote.GeoSocialApi
import com.example.geosocial.data.remote.dto.CreatePostRequest
import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.repository.PostRepository
import javax.inject.Inject

class PostRepositoryImpl @Inject constructor(
    private val api: GeoSocialApi
) : PostRepository {

    override suspend fun getFeed(limit: Int, offset: Int): Result<List<Post>> =
        safeApiCall { api.getFeed(limit, offset).map { it.toDomain() } }

    override suspend fun getPlacePosts(placeId: Long): Result<List<Post>> =
        safeApiCall { api.getPlacePosts(placeId).map { it.toDomain() } }

    override suspend fun addPost(placeId: Long, text: String, rating: Int?): Result<Post>
=
        safeApiCall { api.addPost(placeId, CreatePostRequest(text, null,
rating)).toDomain() }
}

```

Файл app/src/main/java/com/example/geosocial/data/repository/UserRepositoryImpl.kt

```

package com.example.geosocial.data.repository

import com.example.geosocial.data.mapper.toDomain
import com.example.geosocial.data.remote.GeoSocialApi
import com.example.geosocial.data.remote.dto.UpdateProfileRequest
import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.model.User
import com.example.geosocial.domain.repository.UserRepository
import javax.inject.Inject

class UserRepositoryImpl @Inject constructor(
    private val api: GeoSocialApi
) : UserRepository {

    override suspend fun getMe(): Result<User> = safeApiCall { api.getMe().toDomain() }

    override suspend fun updateProfile(displayName: String?, bio: String?): Result<User>
=
        safeApiCall { api.updateProfile(UpdateProfileRequest(displayName,
bio)).toDomain() }

    override suspend fun getUserPlaces(userId: Long): Result<List<Place>> =
        safeApiCall { api.getUserPlaces(userId).map { it.toDomain() } }
}

```

Файл app/src/main/java/com/example/geosocial/di/AppModule.kt

```
package com.example.geosocial.di

import com.example.geosocial.BuildConfig
import com.example.geosocial.data.remote.AuthInterceptor
import com.example.geosocial.data.remote.GeoSocialApi
import com.google.firebase.auth.FirebaseAuth
import com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import kotlinx.serialization.json.Json
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import java.util.concurrent.TimeUnit
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideFirebaseAuth(): FirebaseAuth = FirebaseAuth.getInstance()

    @Provides
    @Singleton
    fun provideJson(): Json = Json {
        ignoreUnknownKeys = true
        encodeDefaults = true
    }

    @Provides
    @Singleton
    fun provideOkHttpClient(authInterceptor: AuthInterceptor): OkHttpClient {
        val logging = HttpLoggingInterceptor().apply {
            level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY
            else HttpLoggingInterceptor.Level.NONE
        }
        return OkHttpClient.Builder()
            .addInterceptor(authInterceptor)
            .addInterceptor(logging)
            .connectTimeout(30, TimeUnit.SECONDS)
            .readTimeout(30, TimeUnit.SECONDS)
            .build()
    }

    @Provides
    @Singleton
    fun provideRetrofit(client: OkHttpClient, json: Json): Retrofit = Retrofit.Builder()
        .baseUrl(BuildConfig.BASE_URL)
        .client(client)
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()

    @Provides
    @Singleton
    fun provideApi(retrofit: Retrofit): GeoSocialApi =
        retrofit.create(GeoSocialApi::class.java)
}
```

Файл app/src/main/java/com/example/geosocial/di/RepositoryModule.kt

```
package com.example.geosocial.di

import com.example.geosocial.data.repository.AuthRepositoryImpl
```

```

import com.example.geosocial.data.repository.PlaceRepositoryImpl
import com.example.geosocial.data.repository.PostRepositoryImpl
import com.example.geosocial.data.repository.UserRepositoryImpl
import com.example.geosocial.domain.repository.AuthRepository
import com.example.geosocial.domain.repository.PlaceRepository
import com.example.geosocial.domain.repository.PostRepository
import com.example.geosocial.domain.repository.UserRepository
import dagger.Binds
import dagger.Module
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindAuthRepository(impl: AuthRepositoryImpl): AuthRepository

    @Binds
    @Singleton
    abstract fun bindUserRepository(impl: UserRepositoryImpl): UserRepository

    @Binds
    @Singleton
    abstract fun bindPlaceRepository(impl: PlaceRepositoryImpl): PlaceRepository

    @Binds
    @Singleton
    abstract fun bindPostRepository(impl: PostRepositoryImpl): PostRepository
}

```

Файл app/src/main/java/com/example/geosocial/domain/model/Models.kt

```

package com.example.geosocial.domain.model

```

```

/** Пользователь социальной сети. */
data class User(
    val id: Long,
    val email: String,
    val displayName: String,
    val bio: String? = null,
    val avatarUrl: String? = null
)

/** Место на карте. */
data class Place(
    val id: Long,
    val title: String,
    val description: String,
    val category: String,
    val latitude: Double,
    val longitude: Double,
    val address: String? = null,
    val authorId: Long,
    val authorName: String,
    val postsCount: Long,
    val distanceKm: Double? = null,
    val createdAt: String
)

/** Пост (отзыв/чек-ин) в месте. */
data class Post(
    val id: Long,
    val placeId: Long,
    val placeTitle: String,
    val text: String,

```

```

        val photoUrl: String? = null,
        val rating: Int? = null,
        val authorId: Long,
        val authorName: String,
        val authorAvatar: String? = null,
        val createdAt: String
    )

    /** Категории мест. */
    object PlaceCategories {
        val ALL = listOf("Кафе", "Парк", "Вид", "Спорт", "Культура", "Другое")
    }

```

Файл `app/src/main/java/com/example/geosocial/domain/repository/Repositories.kt`

```

package com.example.geosocial.domain.repository

import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.model.User
import kotlinx.coroutines.flow.Flow

/** Аутентификация (Firebase) + синхронизация профиля с сервером. */
interface AuthRepository {
    val isLoggedIn: Flow<Boolean>
    suspend fun signIn(email: String, password: String): Result<Unit>
    suspend fun signUp(email: String, password: String, displayName: String):
Result<Unit>
    fun signOut()
}

interface UserRepository {
    suspend fun getMe(): Result<User>
    suspend fun updateProfile(displayName: String?, bio: String?): Result<User>
    suspend fun getUserPlaces(userId: Long): Result<List<Place>>
}

interface PlaceRepository {
    suspend fun getPlaces(
        latitude: Double? = null,
        longitude: Double? = null,
        radiusKm: Double = 10.0,
        query: String? = null,
        category: String? = null
    ): Result<List<Place>>

    suspend fun getPlace(id: Long): Result<Place>

    suspend fun createPlace(
        title: String,
        description: String,
        category: String,
        latitude: Double,
        longitude: Double,
        address: String?
    ): Result<Place>

    suspend fun deletePlace(id: Long): Result<Unit>
}

interface PostRepository {
    suspend fun getFeed(limit: Int = 20, offset: Int = 0): Result<List<Post>>
    suspend fun getPlacePosts(placeId: Long): Result<List<Post>>
    suspend fun addPost(placeId: Long, text: String, rating: Int?): Result<Post>
}

```

Файл app/src/main/java/com/example/geosocial/domain/usecase/UseCases.kt

```
package com.example.geosocial.domain.usecase

import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.model.User
import com.example.geosocial.domain.repository.AuthRepository
import com.example.geosocial.domain.repository.PlaceRepository
import com.example.geosocial.domain.repository.PostRepository
import com.example.geosocial.domain.repository.UserRepository
import javax.inject.Inject

// ----- Аутентификация -----

class SignInUseCase @Inject constructor(private val repository: AuthRepository) {
    suspend operator fun invoke(email: String, password: String): Result<Unit> {
        if (email.isBlank() || password.isBlank()) {
            return Result.failure(IllegalArgumentException("Заполните email и пароль"))
        }
        return repository.signIn(email.trim(), password)
    }
}

class SignUpUseCase @Inject constructor(private val repository: AuthRepository) {
    suspend operator fun invoke(email: String, password: String, name: String):
    Result<Unit> {
        if (email.isBlank() || password.length < 6) {
            return Result.failure(IllegalArgumentException("Пароль должен быть не короче
6 символов"))
        }
        return repository.signUp(email.trim(), password, name.trim())
    }
}

class SignOutUseCase @Inject constructor(private val repository: AuthRepository) {
    operator fun invoke() = repository.signOut()
}

class ObserveAuthStateUseCase @Inject constructor(private val repository: AuthRepository) {
    operator fun invoke() = repository.isLoggedIn
}

// ----- Места -----

class GetPlacesUseCase @Inject constructor(private val repository: PlaceRepository) {
    suspend operator fun invoke(
        latitude: Double? = null,
        longitude: Double? = null,
        radiusKm: Double = 10.0,
        query: String? = null,
        category: String? = null
    ): Result<List<Place>> = repository.getPlaces(latitude, longitude, radiusKm, query,
category)
}

class GetPlaceUseCase @Inject constructor(private val repository: PlaceRepository) {
    suspend operator fun invoke(id: Long): Result<Place> = repository.getPlace(id)
}

class CreatePlaceUseCase @Inject constructor(private val repository: PlaceRepository) {
    suspend operator fun invoke(
        title: String,
        description: String,
        category: String,
        latitude: Double,
        longitude: Double,
        address: String?
    ): Result<Place> {
```

```

        if (title.isBlank()) return Result.failure(IllegalArgumentException("Введите
название места"))
        if (description.isBlank()) return
Result.failure(IllegalArgumentException("Введите описание"))
        return repository.createPlace(title, description, category, latitude, longitude,
address)
    }
}

class DeletePlaceUseCase @Inject constructor(private val repository: PlaceRepository) {
    suspend operator fun invoke(id: Long): Result<Unit> = repository.deletePlace(id)
}

// ----- Посты -----

class GetFeedUseCase @Inject constructor(private val repository: PostRepository) {
    suspend operator fun invoke(): Result<List<Post>> = repository.getFeed()
}

class GetPlacePostsUseCase @Inject constructor(private val repository: PostRepository) {
    suspend operator fun invoke(placeId: Long): Result<List<Post>> =
repository.getPlacePosts(placeId)
}

class AddPostUseCase @Inject constructor(private val repository: PostRepository) {
    suspend operator fun invoke(placeId: Long, text: String, rating: Int?): Result<Post>
{
        if (text.isBlank()) return Result.failure(IllegalArgumentException("Введите текст
поста"))
        return repository.addPost(placeId, text, rating)
    }
}

// ----- Профиль -----

class GetProfileUseCase @Inject constructor(private val repository: UserRepository) {
    suspend operator fun invoke(): Result<User> = repository.getMe()
}

class UpdateProfileUseCase @Inject constructor(private val repository: UserRepository) {
    suspend operator fun invoke(displayName: String?, bio: String?): Result<User> =
        repository.updateProfile(displayName, bio)
}

class GetUserPlacesUseCase @Inject constructor(private val repository: UserRepository) {
    suspend operator fun invoke(userId: Long): Result<List<Place>> =
repository.getUserPlaces(userId)
}

```

Файл app/src/main/java/com/example/geosocial/domain/util/Resource.kt

```

package com.example.geosocial.domain.util

/** Состояние асинхронной операции для UI. */
sealed interface Resource<out T> {
    data object Loading : Resource<Nothing>
    data class Success<T>(val data: T) : Resource<T>
    data class Error(val message: String) : Resource<Nothing>
}

```

Файл app/src/main/java/com/example/geosocial/presentation/auth/AuthViewModel.kt

```

package com.example.geosocial.presentation.auth

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import com.example.geosocial.domain.usecase.ObserveAuthStateUseCase
import com.example.geosocial.domain.usecase.SignInUseCase

```

```

import com.example.geosocial.domain.usecase.SignUpUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch
import javax.inject.Inject

data class AuthUiState(
    val email: String = "",
    val password: String = "",
    val name: String = "",
    val isLoading: Boolean = false,
    val error: String? = null
)

@HiltViewModel
class AuthViewModel @Inject constructor(
    observeAuthState: ObserveAuthStateUseCase,
    private val signIn: SignInUseCase,
    private val signUp: SignUpUseCase
) : ViewModel() {

    /** null – состояние ещё не определено. */
    val isLoggedIn: StateFlow<Boolean?> = observeAuthState()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), null)

    private val _uiState = MutableStateFlow(AuthUiState())
    val uiState: StateFlow<AuthUiState> = _uiState.asStateFlow()

    fun onEmailChange(value: String) { _uiState.value = _uiState.value.copy(email = value, error = null) }
    fun onPasswordChange(value: String) { _uiState.value = _uiState.value.copy(password = value, error = null) }
    fun onNameChange(value: String) { _uiState.value = _uiState.value.copy(name = value, error = null) }

    fun login() {
        val state = _uiState.value
        viewModelScope.launch {
            _uiState.value = state.copy(isLoading = true, error = null)
            val result = signIn(state.email, state.password)
            _uiState.value = _uiState.value.copy(
                isLoading = false,
                error = result.exceptionOrNull()?.message
            )
        }
    }

    fun register() {
        val state = _uiState.value
        viewModelScope.launch {
            _uiState.value = state.copy(isLoading = true, error = null)
            val result = signUp(state.email, state.password, state.name)
            _uiState.value = _uiState.value.copy(
                isLoading = false,
                error = result.exceptionOrNull()?.message
            )
        }
    }
}

```

Файл app/src/main/java/com/example/geosocial/presentation/auth/LoginScreen.kt

```

package com.example.geosocial.presentation.auth

import androidx.compose.foundation.layout.*

```

```

import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp

@Composable
fun LoginScreen(
    viewModel: AuthViewModel,
    onNavigateToRegister: () -> Unit
) {
    val state by viewModel.uiState.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("GeoSocial", style = MaterialTheme.typography.headlineLarge)
        Spacer(Modifier.height(4.dp))
        Text(
            "Социальная сеть местоположений",
            style = MaterialTheme.typography.bodyMedium,
            textAlign = TextAlign.Center
        )
        Spacer(Modifier.height(32.dp))

        OutlinedTextField(
            value = state.email,
            onValueChange = viewModel::onEmailChange,
            label = { Text("Email") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(Modifier.height(12.dp))
        OutlinedTextField(
            value = state.password,
            onValueChange = viewModel::onPasswordChange,
            label = { Text("Пароль") },
            singleLine = true,
            visualTransformation = PasswordVisualTransformation(),
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
            modifier = Modifier.fillMaxWidth()
        )

        state.error?.let {
            Spacer(Modifier.height(8.dp))
            Text(it, color = MaterialTheme.colorScheme.error, style =
MaterialTheme.typography.bodySmall)
        }

        Spacer(Modifier.height(24.dp))
        Button(
            onClick = viewModel::login,
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        ) {
            if (state.isLoading) {

```

```

                CircularProgressIndicator(modifier = Modifier.size(20.dp), strokeWidth =
2.dp)
            } else {
                Text("Войти")
            }
        }
        Spacer(Modifier.height(8.dp))
        TextButton(onClick = onNavigateToRegister) {
            Text("Нет аккаунта? Зарегистрироваться")
        }
    }
}

```

Файл app/src/main/java/com/example/geosocial/presentation/auth/RegisterScreen.kt

```

package com.example.geosocial.presentation.auth

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.unit.dp

@Composable
fun RegisterScreen(
    viewModel: AuthViewModel,
    onNavigateBack: () -> Unit
) {
    val state by viewModel.uiState.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Регистрация", style = MaterialTheme.typography.headlineMedium)
        Spacer(Modifier.height(24.dp))

        OutlinedTextField(
            value = state.name,
            onValueChange = viewModel::onNameChange,
            label = { Text("Имя") },
            singleLine = true,
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(Modifier.height(12.dp))
        OutlinedTextField(
            value = state.email,
            onValueChange = viewModel::onEmailChange,
            label = { Text("Email") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(Modifier.height(12.dp))
        OutlinedTextField(
            value = state.password,
            onValueChange = viewModel::onPasswordChange,
            label = { Text("Пароль (мин. 6 символов)") },

```

```

        singleLine = true,
        visualTransformation = PasswordVisualTransformation(),
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
        modifier = Modifier.fillMaxWidth()
    )

    state.error?.let {
        Spacer(Modifier.height(8.dp))
        Text(it, color = MaterialTheme.colorScheme.error, style =
MaterialTheme.typography.bodySmall)
    }

    Spacer(Modifier.height(24.dp))
    Button(
        onClick = viewModel::register,
        enabled = !state.isLoading,
        modifier = Modifier.fillMaxWidth()
    ) {
        if (state.isLoading) {
            CircularProgressIndicator(modifier = Modifier.size(20.dp), strokeWidth =
2.dp)
        } else {
            Text("Создать аккаунт")
        }
    }
    Spacer(Modifier.height(8.dp))
    TextButton(onClick = onNavigateBack) { Text("Уже есть аккаунт? Войти") }
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/components/Components.kt

```

package com.example.geosocial.presentation.components

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Place
import androidx.compose.material.icons.filled.Star
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.model.Post

@Composable
fun PlaceCard(place: Place, onClick: () -> Unit, modifier: Modifier = Modifier) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
    ) {
        Column(Modifier.padding(16.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Filled.Place, contentDescription = null, tint =
MaterialTheme.colorScheme.primary)
                Spacer(Modifier.width(8.dp))
                Text(
                    place.title,
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.SemiBold,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,

```

```

        modifier = Modifier.weight(1f)
    )
    AssistChip(onClick = {}, label = { Text(place.category) })
}
Spacer(Modifier.height(8.dp))
Text(
    place.description,
    style = MaterialTheme.typography.bodyMedium,
    maxLines = 2,
    overflow = TextOverflow.Ellipsis
)
Spacer(Modifier.height(8.dp))
Row(verticalAlignment = Alignment.CenterVertically) {
    Text(
        "Автор: ${place.authorName}",
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(Modifier.weight(1f))
    place.distanceKm?.let {
        Text(
            String.format("%.1f км", it),
            style = MaterialTheme.typography.labelSmall,
            color = MaterialTheme.colorScheme.primary
        )
        Spacer(Modifier.width(12.dp))
    }
    Text(
        "Постов: ${place.postsCount}",
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
}

@Composable
fun PostCard(post: Post, onPlaceClick: (Long) -> Unit, modifier: Modifier = Modifier) {
    Card(modifier = modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Box(
                    modifier = Modifier
                        .size(36.dp)
                        .clip(CircleShape)
                        .background(MaterialTheme.colorScheme.primaryContainer),
                    contentAlignment = Alignment.Center
                ) {
                    Text(post.authorName.take(1).uppercase())
                }
                Spacer(Modifier.width(10.dp))
                Column(Modifier.weight(1f)) {
                    Text(post.authorName, style = MaterialTheme.typography.titleSmall)
                    Text(
                        post.placeTitle,
                        style = MaterialTheme.typography.labelSmall,
                        color = MaterialTheme.colorScheme.primary,
                        modifier = Modifier.clickable { onPlaceClick(post.placeId) }
                    )
                }
            }
            post.rating?.let { rating ->
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(
                        Icons.Filled.Star,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.tertiary,
                        modifier = Modifier.size(16.dp)
                    )
                    Text("$rating", style = MaterialTheme.typography.labelMedium)
                }
            }
        }
    }
}

```

```

        }
    }
    Spacer(Modifier.height(10.dp))
    Text(post.text, style = MaterialTheme.typography.bodyMedium)
    Spacer(Modifier.height(6.dp))
    Text(
        post.createdAt.replace("T", " ").take(16),
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
}

@Composable
fun LoadingBox(modifier: Modifier = Modifier) {
    Box(modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        CircularProgressIndicator()
    }
}

@Composable
fun MessageBox(text: String, modifier: Modifier = Modifier) {
    Box(modifier.fillMaxSize().padding(24.dp), contentAlignment = Alignment.Center) {
        Text(text, style = MaterialTheme.typography.bodyLarge)
    }
}
}

```

Файл

app/src/main/java/com/example/geosocial/presentation/createplace/CreatePlaceScreen.kt

```

package com.example.geosocial.presentation.createplace

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.example.geosocial.domain.model.PlaceCategories
import com.google.android.gms.maps.model.CameraPosition
import com.google.android.gms.maps.model.LatLng
import com.google.maps.android.compose.GoogleMap
import com.google.maps.android.compose.Marker
import com.google.maps.android.compose.rememberMarkerState
import com.google.maps.android.compose.rememberCameraPositionState

@OptIn(ExperimentalMaterial3Api::class, ExperimentalLayoutApi::class)
@Composable
fun CreatePlaceScreen(
    onDone: () -> Unit,
    viewModel: CreatePlaceViewModel = hiltViewModel()
) {
    val state by viewModel.uiState.collectAsState()

    LaunchedEffect(state.isSaved) {
        if (state.isSaved) onDone()
    }

    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(LatLng(state.latitude, state.longitude),
15f)
    }
    val markerState = rememberMarkerState(position = LatLng(state.latitude,
state.longitude))
}

```

```

Scaffold(
  topBar = {
    TopAppBar(
      title = { Text("Новое место") },
      navigationIcon = {
        IconButton(onClick = onDone) {
          Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Назад")
        }
      }
    )
  }
) { padding ->
  Column(
    Modifier
      .padding(padding)
      .fillMaxSize()
      .verticalScroll(rememberScrollState())
      .padding(16.dp)
  ) {
    Text("Нажмите на карту, чтобы уточнить точку", style =
MaterialTheme.typography.labelMedium)
    Spacer(Modifier.height(8.dp))
    Box(
      Modifier
        .fillMaxWidth()
        .height(220.dp)
    ) {
      GoogleMap(
        modifier = Modifier.fillMaxSize(),
        cameraPositionState = cameraPositionState,
        onMapClick = { latLng ->
          markerState.position = latLng
          viewModel.onLocationChange(latLng.latitude, latLng.longitude)
        }
      ) {
        Marker(state = markerState, title = "Новое место")
      }
    }
    Spacer(Modifier.height(8.dp))
    Text(
      "Координаты: ${"% .5f".format(state.latitude)},
${"% .5f".format(state.longitude)}",
      style = MaterialTheme.typography.labelSmall
    )

    Spacer(Modifier.height(16.dp))
    OutlinedTextField(
      value = state.title,
      onValueChange = viewModel::onTitleChange,
      label = { Text("Название") },
      singleLine = true,
      modifier = Modifier.fillMaxWidth()
    )
    Spacer(Modifier.height(12.dp))
    OutlinedTextField(
      value = state.description,
      onValueChange = viewModel::onDescriptionChange,
      label = { Text("Описание") },
      minLines = 3,
      modifier = Modifier.fillMaxWidth()
    )
    Spacer(Modifier.height(12.dp))
    OutlinedTextField(
      value = state.address,
      onValueChange = viewModel::onAddressChange,
      label = { Text("Адрес (необязательно)") },
      singleLine = true,
      modifier = Modifier.fillMaxWidth()
    )
  }
}

```

```

        Spacer(Modifier.height(12.dp))
        Text("Категория", style = MaterialTheme.typography.labelLarge)
        FlowRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            PlaceCategories.ALL.forEach { category ->
                FilterChip(
                    selected = state.category == category,
                    onClick = { viewModel.onCategoryChange(category) },
                    label = { Text(category) }
                )
            }
        }

        state.error?.let {
            Spacer(Modifier.height(8.dp))
            Text(it, color = MaterialTheme.colorScheme.error,
                style = MaterialTheme.typography.bodySmall)
        }

        Spacer(Modifier.height(20.dp))
        Button(
            onClick = viewModel::save,
            enabled = !state.isSaving,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(if (state.isSaving) "Сохранение..." else "Сохранить место")
        }
        Spacer(Modifier.height(24.dp))
    }
}

```

Файл

app/src/main/java/com/example/geosocial/presentation/createplace/CreatePlaceViewModel.kt

```

package com.example.geosocial.presentation.createplace

import androidx.lifecycle.SavedStateHandle
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.geosocial.domain.model.PlaceCategories
import com.example.geosocial.domain.usecase.CreatePlaceUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class CreatePlaceUiState(
    val title: String = "",
    val description: String = "",
    val address: String = "",
    val category: String = PlaceCategories.ALL.first(),
    val latitude: Double = 55.7558,
    val longitude: Double = 37.6173,
    val isSaving: Boolean = false,
    val error: String? = null,
    val isSaved: Boolean = false
)

@HiltViewModel
class CreatePlaceViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val createPlace: CreatePlaceUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(
        CreatePlaceUiState()
    )
}

```

```

        latitude = (savedStateHandle.get<Float>("lat") ?: 55.7558f).toDouble(),
        longitude = (savedStateHandle.get<Float>("lng") ?: 37.6173f).toDouble()
    )
)
val uiState: StateFlow<CreatePlaceUiState> = _uiState.asStateFlow()

fun onTitleChange(value: String) { _uiState.value = _uiState.value.copy(title =
value) }
fun onDescriptionChange(value: String) { _uiState.value =
_uiState.value.copy(description = value) }
fun onAddressChange(value: String) { _uiState.value = _uiState.value.copy(address =
value) }
fun onCategoryChange(value: String) { _uiState.value = _uiState.value.copy(category =
value) }

fun onLocationChange(latitude: Double, longitude: Double) {
    _uiState.value = _uiState.value.copy(latitude = latitude, longitude = longitude)
}

fun save() {
    val state = _uiState.value
    viewModelScope.launch {
        _uiState.value = state.copy(isSaving = true, error = null)
        createPlace(
            title = state.title,
            description = state.description,
            category = state.category,
            latitude = state.latitude,
            longitude = state.longitude,
            address = state.address.takeIf { it.isNotBlank() }
        )
        .onSuccess { _uiState.value = _uiState.value.copy(isSaving = false,
isSaved = true) }
        .onFailure {
            _uiState.value = _uiState.value.copy(isSaving = false, error =
it.message)
        }
    }
}
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/feed/FeedScreen.kt

```

package com.example.geosocial.presentation.feed

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Refresh
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.example.geosocial.presentation.components.LoadingBox
import com.example.geosocial.presentation.components.MessageBox
import com.example.geosocial.presentation.components.PostCard

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun FeedScreen(
    onPlaceClick: (Long) -> Unit,
    viewModel: FeedViewModel = hiltViewModel()
) {
    val state by viewModel.uiState.collectAsState()

    Scaffold(

```

```

        topBar = {
            TopAppBar(
                title = { Text("Лента") },
                actions = {
                    IconButton(onClick = viewModel::load) {
                        Icon(Icons.Filled.Refresh, contentDescription = "Обновить")
                    }
                }
            )
        }
    }
) { padding ->
    Box(Modifier.padding(padding)) {
        when {
            state.isLoading -> LoadingBox()
            state.error != null -> MessageBox(state.error!!)
            state.posts.isEmpty() -> MessageBox(
                "Пока нет ни одного поста.\nОткройте карту и добавьте первое место!"
            )
            else -> LazyColumn(
                contentPadding = PaddingValues(16.dp),
                verticalArrangement = Arrangement.spacedBy(12.dp)
            ) {
                items(state.posts, key = { it.id }) { post ->
                    PostCard(post = post, onPlaceClick = onPlaceClick)
                }
            }
        }
    }
}
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/feed/FeedViewModel.kt

```
package com.example.geosocial.presentation.feed
```

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.usecase.GetFeedUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class FeedUiState(
    val posts: List<Post> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null
)

@HiltViewModel
class FeedViewModel @Inject constructor(
    private val getFeed: GetFeedUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(FeedUiState())
    val uiState: StateFlow<FeedUiState> = _uiState.asStateFlow()

    init { load() }

    fun load() {
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(isLoading = true, error = null)
            getFeed()
                .onSuccess { _uiState.value = FeedUiState(posts = it, isLoading = false) }
                .onFailure {

```

```

        _uiState.value = _uiState.value.copy(isLoading = false, error =
it.message)
    }
}
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/map/MapScreen.kt

```

package com.example.geosocial.presentation.map

import android.Manifest
import android.annotation.SuppressLint
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.core.content.ContextCompat
import androidx.hilt.navigation.compose.hiltViewModel
import com.google.android.gms.location.LocationServices
import com.google.android.gms.maps.model.CameraPosition
import com.google.android.gms.maps.model.LatLng
import com.google.maps.android.compose.*
import kotlinx.coroutines.tasks.await

private val MOSCOW = LatLng(55.7558, 37.6173)

@OptIn(ExperimentalMaterial3Api::class)
@SuppressLint("MissingPermission")
@Composable
fun MapScreen(
    onPlaceClick: (Long) -> Unit,
    onAddPlace: (Double, Double) -> Unit,
    viewModel: MapViewModel = hiltViewModel()
) {
    val state by viewModel.uiState.collectAsState()
    val context = LocalContext.current

    var hasLocationPermission by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(context,
Manifest.permission.ACCESS_FINE_LOCATION) ==
PackageManager.PERMISSION_GRANTED
        )
    }
    val permissionLauncher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted -> hasLocationPermission = granted }

    LaunchedEffect(Unit) {
        if (!hasLocationPermission) {
            permissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)
        }
    }

    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(MOSCOW, 11f)
    }

    // Определяем текущее местоположение и подгружаем места рядом
    LaunchedEffect(hasLocationPermission) {

```

```

        if (hasLocationPermission) {
            runCatching {
                LocationServices.getFusedLocationProviderClient(context).lastLocation.await()
                    ?.getOrNull()?.let { location ->
                        viewModel.onUserLocation(location.latitude, location.longitude)
                        cameraPositionState.position = CameraPosition.fromLatLngZoom(
                            LatLng(location.latitude, location.longitude), 13f
                        )
                    }
            }
        }

        Scaffold(
            topBar = {
                TopAppBar(title = { Text("Места на карте") })
            },
            floatingActionButton = {
                ExtendedFloatingActionButton(
                    onClick = {
                        val target = cameraPositionState.position.target
                        onAddPlace(target.latitude, target.longitude)
                    },
                    icon = { Icon(Icons.Filled.Add, contentDescription = null) },
                    text = { Text("Добавить место") }
                )
            }
        ) { padding ->
            Column(Modifier.padding(padding).fillMaxSize()) {

                OutlinedTextField(
                    value = state.query,
                    onValueChange = viewModel::onQueryChange,
                    label = { Text("Поиск по названию") },
                    singleLine = true,
                    trailingIcon = {
                        TextButton(onClick = viewModel::load) { Text("Найти") }
                    },
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 8.dp)
                )

                Box(Modifier.weight(1f)) {
                    GoogleMap(
                        modifier = Modifier.fillMaxSize(),
                        cameraPositionState = cameraPositionState,
                        properties = MapProperties(isMyLocationEnabled =
hasLocationPermission),
                        uiSettings = MapUiSettings(zoomControlsEnabled = false),
                        onMapLongClick = { latLng -> onAddPlace(latLng.latitude,
latLng.longitude) }
                    ) {
                        state.places.forEach { place ->
                            Marker(
                                state = rememberMarkerState(
                                    key = place.id.toString(),
                                    position = LatLng(place.latitude, place.longitude)
                                ),
                                title = place.title,
                                snippet = "${place.category} · постов: ${place.postsCount}",
                                onInfoWindowClick = { onPlaceClick(place.id) }
                            )
                        }
                    }
                }

                if (state.isLoading) {
                    CircularProgressIndicator(Modifier.align(Alignment.Center))
                }
                state.error?.let {
                    Surface(

```

```

        color = MaterialTheme.colorScheme.errorContainer,
        modifier = Modifier.align(Alignment.BottomCenter).fillMaxWidth()
    ) {
        Text(it, Modifier.padding(12.dp))
    }
}

Text(
    "Долгое нажатие на карте – добавить место в этой точке",
    style = MaterialTheme.typography.labelSmall,
    modifier = Modifier.padding(horizontal = 16.dp, vertical = 6.dp)
)
}
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/map/MapViewModel.kt

```

package com.example.geosocial.presentation.map

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.geosocial.domain.model.Place
import com.example.geosocial.domain.usecase.GetPlacesUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class MapUiState(
    val places: List<Place> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null,
    val userLatitude: Double? = null,
    val userLongitude: Double? = null,
    val radiusKm: Double = 10.0,
    val query: String = ""
)

@HiltViewModel
class MapViewModel @Inject constructor(
    private val getPlaces: GetPlacesUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(MapUiState())
    val uiState: StateFlow<MapUiState> = _uiState.asStateFlow()

    init { load() }

    fun onUserLocation(latitude: Double, longitude: Double) {
        _uiState.value = _uiState.value.copy(userLatitude = latitude, userLongitude = longitude)
        load()
    }

    fun onQueryChange(value: String) {
        _uiState.value = _uiState.value.copy(query = value)
    }

    fun load() {
        val state = _uiState.value
        viewModelScope.launch {
            _uiState.value = state.copy(isLoading = true, error = null)
            getPlaces(
                latitude = state.userLatitude,
                longitude = state.userLongitude,
            )
        }
    }
}

```

```

        radiusKm = state.radiusKm,
        query = state.query.takeIf { it.isNotBlank() }
    )
    .onSuccess {
        _uiState.value = _uiState.value.copy(places = it, isLoading = false)
    }
    .onFailure {
        _uiState.value = _uiState.value.copy(isLoading = false, error =
it.message)
    }
}
}
}
}

```

Файл `app/src/main/java/com/example/geosocial/presentation/navigation/AppNavigation.kt`

```

package com.example.geosocial.presentation.navigation

import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Map
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.automirrored.filled.List
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Icon
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.hilt.navigation.compose.hiltViewModel
import androidx.navigation.NavDestination.Companion.hierarchy
import androidx.navigation.NavGraph.Companion.findStartDestination
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.geosocial.presentation.auth.AuthViewModel
import com.example.geosocial.presentation.auth.LoginScreen
import com.example.geosocial.presentation.auth.RegisterScreen
import com.example.geosocial.presentation.createplace.CreatePlaceScreen
import com.example.geosocial.presentation.feed.FeedScreen
import com.example.geosocial.presentation.map.MapScreen
import com.example.geosocial.presentation.place.PlaceDetailScreen
import com.example.geosocial.presentation.profile.ProfileScreen

private data class BottomTab(val route: String, val label: String, val icon: ImageVector)

@Composable
fun AppNavigation() {
    val authViewModel: AuthViewModel = hiltViewModel()
    val isLoggedIn by authViewModel.isLoggedIn.collectAsState()

    when (isLoggedIn) {
        // состояние авторизации ещё не известно
        null -> Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
        false -> AuthNavHost(authViewModel)
        true -> MainNavHost()
    }
}

```

```

}

@Composable
private fun AuthNavHost(authViewModel: AuthViewModel) {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = Routes.LOGIN) {
        composable(Routes.LOGIN) {
            LoginScreen(
                viewModel = authViewModel,
                onNavigateToRegister = { navController.navigate(Routes.REGISTER) }
            )
        }
        composable(Routes.REGISTER) {
            RegisterScreen(
                viewModel = authViewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }
    }
}

@Composable
private fun MainNavHost() {
    val navController = rememberNavController()
    val tabs = listOf(
        BottomTab(Routes.FEED, "Лента", Icons.AutoMirrored.Filled.List),
        BottomTab(Routes.MAP, "Карта", Icons.Filled.Map),
        BottomTab(Routes.PROFILE, "Профиль", Icons.Filled.Person)
    )
    val backStackEntry by navController.currentBackStackEntryAsState()
    val currentDestination = backStackEntry?.destination
    val showBottomBar = tabs.any { tab ->
        currentDestination?.hierarchy?.any { it.route == tab.route } == true
    }

    Scaffold(
        bottomBar = {
            if (showBottomBar) {
                NavigationBar {
                    tabs.forEach { tab ->
                        NavigationBarItem(
                            selected = currentDestination?.hierarchy?.any { it.route ==
tab.route } == true,
                            onClick = {
                                navController.navigate(tab.route) {
                                    popUpTo(navController.graph.findStartDestination().id) {
                                        saveState = true
                                    }
                                    launchSingleTop = true
                                    restoreState = true
                                }
                            },
                            icon = { Icon(tab.icon, contentDescription = tab.label) },
                            label = { Text(tab.label) }
                        )
                    }
                }
            }
        }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = Routes.FEED,
            modifier = Modifier.padding(padding)
        ) {
            composable(Routes.FEED) {
                FeedScreen(onPlaceClick = {
                    navController.navigate(Routes.placeDetail(it)) })
            }
            composable(Routes.MAP) {

```

```

        MapScreen(
            onPlaceClick = { navController.navigate(Routes.placeDetail(it)) },
            onAddPlace = { lat, lng ->
navController.navigate(Routes.createPlace(lat, lng)) }
        )
    }
    composable(Routes.PROFILE) {
        ProfileScreen(onPlaceClick = {
navController.navigate(Routes.placeDetail(it)) })
    }
    composable(
        route = Routes.PLACE_DETAIL,
        arguments = listOf(navArgument("placeId") { type = NavType.LongType })
    ) {
        PlaceDetailScreen(onBack = { navController.popBackStack() })
    }
    composable(
        route = Routes.CREATE_PLACE,
        arguments = listOf(
            navArgument("lat") { type = NavType.FloatType; defaultValue =
55.7558f },
            navArgument("lng") { type = NavType.FloatType; defaultValue =
37.6173f }
        )
    ) {
        CreatePlaceScreen(onDone = { navController.popBackStack() })
    }
}
}
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/navigation/Routes.kt

```
package com.example.geosocial.presentation.navigation
```

```

object Routes {
    const val LOGIN = "login"
    const val REGISTER = "register"
    const val FEED = "feed"
    const val MAP = "map"
    const val PROFILE = "profile"
    const val CREATE_PLACE = "create_place?lat={lat}&lng={lng}"
    const val PLACE_DETAIL = "place/{placeId}"

    fun createPlace(lat: Double, lng: Double) = "create_place?lat=$lat&lng=$lng"
    fun placeDetail(placeId: Long) = "place/$placeId"
}

```

Файл app/src/main/java/com/example/geosocial/presentation/place/PlaceDetailScreen.kt

```
package com.example.geosocial.presentation.place
```

```

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Send
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.example.geosocial.presentation.components.LoadingBox
import com.example.geosocial.presentation.components.MessageBox
import com.example.geosocial.presentation.components.PostCard

```

```

@OptIn(ExperimentalMaterial3Api::class, ExperimentalLayoutApi::class)
@Composable
fun PlaceDetailScreen(
    onBack: () -> Unit,
    viewModel: PlaceDetailViewModel = hiltViewModel()
) {
    val state by viewModel.uiState.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(state.place?.title ?: "Место") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Назад")
                    }
                }
            )
        }
    ) { padding ->
        when {
            state.isLoading -> LoadingBox(Modifier.padding(padding))
            state.place == null -> MessageBox(state.error ?: "Место не найдено",
Modifier.padding(padding))
            else -> {
                val place = state.place!!
                LazyColumn(
                    modifier = Modifier.padding(padding).fillMaxSize(),
                    contentPadding = PaddingValues(16.dp),
                    verticalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    item {
                        Card(Modifier.fillMaxWidth()) {
                            Column(Modifier.padding(16.dp)) {
                                Text(place.title, style =
MaterialTheme.typography.headlineSmall)
                                Spacer(Modifier.height(4.dp))
                                Text(place.category, style =
MaterialTheme.typography.labelMedium,
                                    color = MaterialTheme.colorScheme.primary)
                                Spacer(Modifier.height(8.dp))
                                Text(place.description, style =
MaterialTheme.typography.bodyMedium)
                                place.address?.let {
                                    Spacer(Modifier.height(8.dp))
                                    Text(it, style = MaterialTheme.typography.bodySmall)
                                }
                                Spacer(Modifier.height(8.dp))
                                Text(
                                    "${"%5f".format(place.longitude)}",
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    "Координаты: ${"%5f".format(place.latitude)}",
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    "Добавил: ${place.authorName}",
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                    item {
                        Card(Modifier.fillMaxWidth()) {
                            Column(Modifier.padding(16.dp)) {
                                Text("Оставить отзыв", style =
MaterialTheme.typography.titleSmall)
                                Spacer(Modifier.height(8.dp))
                                OutlinedTextField(

```

```

        value = state.newPostText,
        onValueChange = viewModel::onPostTextChange,
        label = { Text("Что здесь интересного?" ) },
        modifier = Modifier.fillMaxWidth(),
        minLines = 2
    )
    Spacer(Modifier.height(8.dp))
    Row(verticalAlignment =
androidx.compose.ui.Alignment.CenterVertically) {
        Text("Оценка:", style =
MaterialTheme.typography.labelMedium)
        Spacer(Modifier.width(8.dp))
        (1..5).forEach { value ->
            FilterChip(
                selected = state.newPostRating == value,
                onClick = {
                    viewModel.onRatingChange(
                        if (state.newPostRating == value)
null else value
                    )
                },
                label = { Text("$value" ) },
                modifier = Modifier.padding(end = 4.dp)
            )
        }
    }
    Spacer(Modifier.height(8.dp))
    Button(
        onClick = viewModel::sendPost,
        enabled = !state.isSending &&
state.newPostText.isNotBlank()
    ) {
        Icon(Icons.Filled.Send, contentDescription = null,
            modifier = Modifier.size(18.dp))
        Spacer(Modifier.width(8.dp))
        Text("Опубликовать")
    }
    state.error?.let {
        Spacer(Modifier.height(8.dp))
        Text(it, color = MaterialTheme.colorScheme.error,
            style = MaterialTheme.typography.bodySmall)
    }
    }
    }
    }
    item {
        Text(
            "Отзывы (${state.posts.size})",
            style = MaterialTheme.typography.titleMedium
        )
    }
    items(state.posts, key = { it.id }) { post ->
        PostCard(post = post, onPlaceClick = {})
    }
    }
    }
    }
    }
}

```

Файл app/src/main/java/com/example/geosocial/presentation/place/PlaceDetailViewModel.kt

```
package com.example.geosocial.presentation.place
```

```

import androidx.lifecycle.SavedStateHandle
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.geosocial.domain.model.Place

```

```

import com.example.geosocial.domain.model.Post
import com.example.geosocial.domain.usecase.AddPostUseCase
import com.example.geosocial.domain.usecase.GetPlacePostsUseCase
import com.example.geosocial.domain.usecase.GetPlaceUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class PlaceDetailUiState(
    val place: Place? = null,
    val posts: List<Post> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null,
    val newPostText: String = "",
    val newPostRating: Int? = null,
    val isSending: Boolean = false
)

@HiltViewModel
class PlaceDetailViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val getPlace: GetPlaceUseCase,
    private val getPlacePosts: GetPlacePostsUseCase,
    private val addPost: AddPostUseCase
) : ViewModel() {

    private val placeId: Long = savedStateHandle.get<Long>("placeId") ?: 0L

    private val _uiState = MutableStateFlow(PlaceDetailUiState())
    val uiState: StateFlow<PlaceDetailUiState> = _uiState.asStateFlow()

    init { load() }

    fun load() {
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(isLoading = true, error = null)
            val placeResult = getPlace(placeId)
            val postsResult = getPlacePosts(placeId)
            _uiState.value = _uiState.value.copy(
                place = placeResult.getOrNull(),
                posts = postsResult.getOrDefault(emptyList()),
                isLoading = false,
                error = placeResult.exceptionOrNull()?.message
            )
        }
    }

    fun onPostTextChange(value: String) {
        _uiState.value = _uiState.value.copy(newPostText = value)
    }

    fun onRatingChange(value: Int?) {
        _uiState.value = _uiState.value.copy(newPostRating = value)
    }

    fun sendPost() {
        val state = _uiState.value
        viewModelScope.launch {
            _uiState.value = state.copy(isSending = true, error = null)
            addPost(placeId, state.newPostText, state.newPostRating)
                .onSuccess { post ->
                    _uiState.value = _uiState.value.copy(
                        posts = listOf(post) + _uiState.value.posts,
                        newPostText = "",
                        newPostRating = null,
                        isSending = false
                    )
                }
        }
    }
}

```

```

        }
        .onFailure {
            _uiState.value = _uiState.value.copy(isSending = false, error =
it.message)
        }
    }
}
}

```

Файл app/src/main/java/com/example/geosocial/presentation/profile/ProfileScreen.kt

```

package com.example.geosocial.presentation.profile

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.Logout
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.example.geosocial.presentation.components.LoadingBox
import com.example.geosocial.presentation.components.MessageBox
import com.example.geosocial.presentation.components.PlaceCard

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProfileScreen(
    onPlaceClick: (Long) -> Unit,
    viewModel: ProfileViewModel = hiltViewModel()
) {
    val state by viewModel.uiState.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Профиль") },
                actions = {
                    IconButton(onClick = viewModel::logout) {
                        Icon(Icons.AutoMirrored.Filled.Logout, contentDescription =
"Выйти")
                    }
                }
            )
        }
    ) { padding ->
        when {
            state.isLoading -> LoadingBox(Modifier.padding(padding))
            state.user == null -> MessageBox(state.error ?: "Не удалось загрузить
профиль",
                Modifier.padding(padding))
            else -> {
                val user = state.user!!
                LazyColumn(
                    modifier = Modifier.padding(padding).fillMaxSize(),
                    contentPadding = PaddingValues(16.dp),
                    verticalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    item {
                        Card(Modifier.fillMaxWidth()) {
                            Column(Modifier.padding(16.dp)) {
                                if (state.isEditing) {
                                    OutlinedTextField(
                                        value = state.editName,
                                        onValueChange = viewModel::onNameChange,
                                        label = { Text("Имя") },

```



```

        val isEditing: Boolean = false,
        val editName: String = "",
        val editBio: String = ""
    )

    @HiltViewModel
    class ProfileViewModel @Inject constructor(
        private val getProfile: GetProfileUseCase,
        private val getUserPlaces: GetUserPlacesUseCase,
        private val updateProfile: UpdateProfileUseCase,
        private val signOut: SignOutUseCase
    ) : ViewModel() {

        private val _uiState = MutableStateFlow(ProfileUiState())
        val uiState: StateFlow<ProfileUiState> = _uiState.asStateFlow()

        init { load() }

        fun load() {
            viewModelScope.launch {
                _uiState.value = _uiState.value.copy(isLoading = true, error = null)
                getProfile()
                    .onSuccess { user ->
                        val places = getUserPlaces(user.id).getOrDefault(emptyList())
                        _uiState.value = _uiState.value.copy(
                            user = user,
                            places = places,
                            isLoading = false,
                            editName = user.displayName,
                            editBio = user.bio.orEmpty()
                        )
                    }
                    .onFailure {
                        _uiState.value = _uiState.value.copy(isLoading = false, error =
it.message)
                    }
            }
        }

        fun startEditing() { _uiState.value = _uiState.value.copy(isEditing = true) }
        fun onNameChange(value: String) { _uiState.value = _uiState.value.copy(editName =
value) }
        fun onBioChange(value: String) { _uiState.value = _uiState.value.copy(editBio =
value) }

        fun saveProfile() {
            val state = _uiState.value
            viewModelScope.launch {
                updateProfile(state.editName, state.editBio)
                    .onSuccess {
                        _uiState.value = _uiState.value.copy(user = it, isEditing = false)
                    }
                    .onFailure { _uiState.value = _uiState.value.copy(error = it.message) }
            }
        }

        fun logout() = signOut()
    }

```

Файл app/src/main/java/com/example/geosocial/ui/theme/Theme.kt

```

package com.example.geosocial.ui.theme

import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Typography
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable

```

```

import androidx.compose.ui.graphics.Color

private val Teal = Color(0xFF00696D)
private val TealLight = Color(0xFF6FF6FC)
private val Coral = Color(0xFFB3261E)

private val LightColors = lightColorScheme(
    primary = Teal,
    secondary = Color(0xFF4A6365),
    tertiary = Color(0xFF4B607C),
    error = Coral
)

private val DarkColors = darkColorScheme(
    primary = TealLight,
    secondary = Color(0xFFB1CBCD),
    tertiary = Color(0xFFB3C8E8)
)

@Composable
fun GeoSocialTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColors else LightColors,
        typography = Typography(),
        content = content
    )
}

```

Файл build.gradle.kts

```

plugins {
    id("com.android.application") version "8.7.2" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.serialization") version "2.0.21" apply false
    id("com.google.devtools.ksp") version "2.0.21-1.0.25" apply false
    id("com.google.dagger.hilt.android") version "2.52" apply false
    id("com.google.gms.google-services") version "4.4.2" apply false
}

```

Файл settings.gradle.kts

```

pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.\\.android.*")
                includeGroupByRegex("com\\.\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "GeoSocial"
include(":app")

```

Приложение В. Экраны мобильного приложения

12:30 87%

GeoSocial

Социальная сеть местоположений

Email
deni@example.com

Пароль
••••••••

Войти

Нет аккаунта? Зарегистрироваться

Рисунок В.1 – Экран входа

12:30 87%

Регистрация

Имя
Дени

Email
deni@example.com

Пароль (мин. 6 символов)
Введите пароль

Создать аккаунт

Уже есть аккаунт? Войти

Рисунок В.2 – Экран регистрации

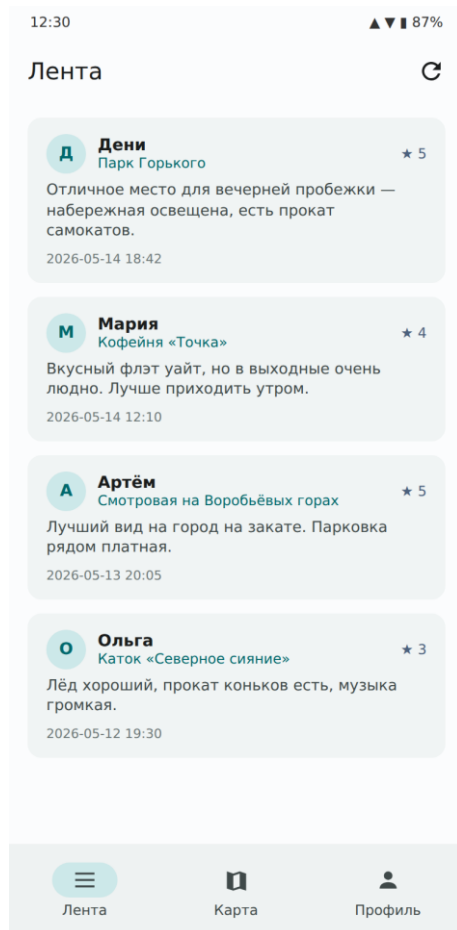


Рисунок В.3 – Экран ленты

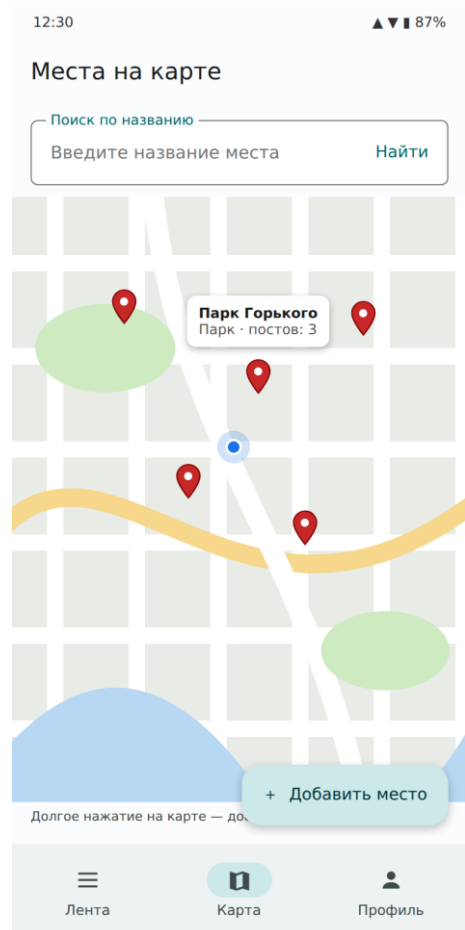


Рисунок В.4 – Экран карты

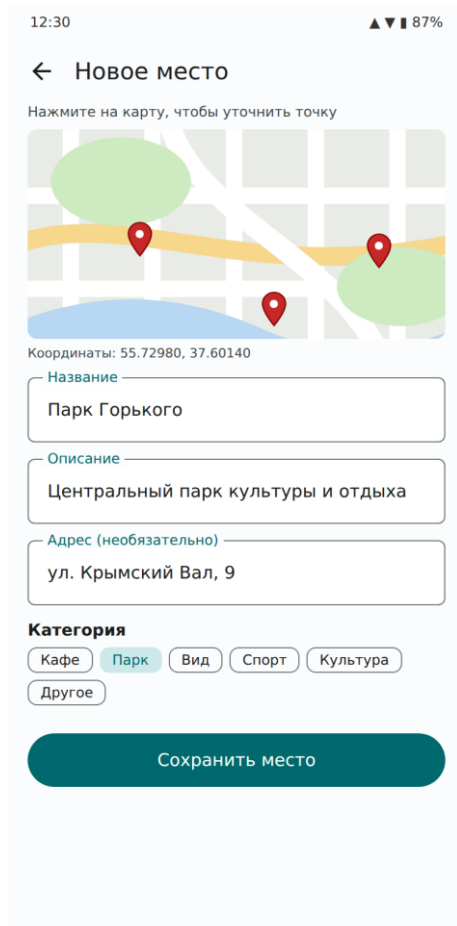


Рисунок В.5 – Экран создания места

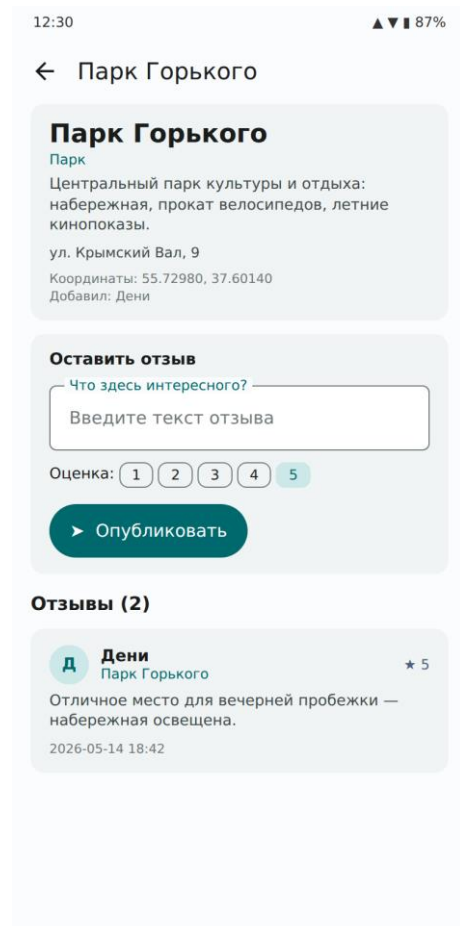


Рисунок В.6 – Экран места

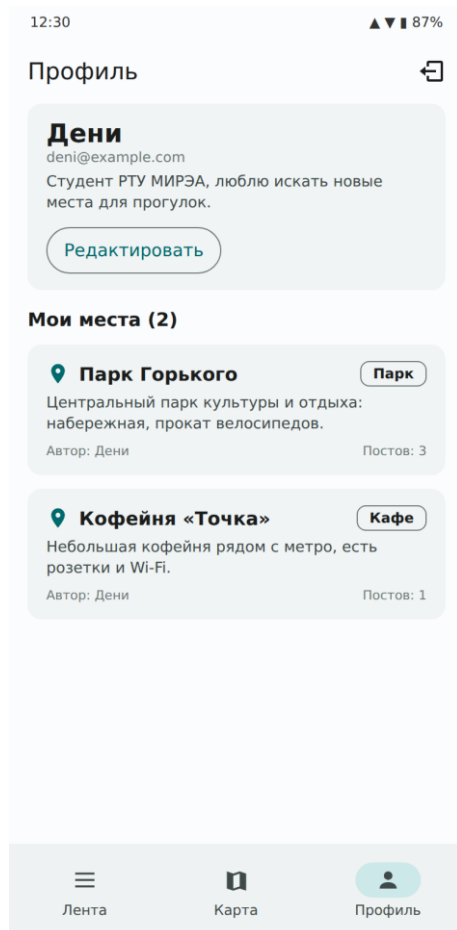


Рисунок В.7 – Экран профиля

Приложение Г. Тестовые сценарии

Таблица Г.1 – Полное описание тестовых сценариев

№	Название	Предусловие	Шаги	Ожидаемый результат	Фактический результат
1	Регистрация нового пользователя	Сервер и БД запущены; пользователь с email test@geo.ru отсутствует	1. Открыть приложение. 2. Нажать «Нет аккаунта? Зарегистрироваться». 3. Ввести имя «Тест», email test@geo.ru, пароль «pass1234». 4. Нажать «Создать аккаунт»	Открыт экран ленты; в таблице users создана запись с email test@geo.ru; в Firebase Console появился пользователь	Соответствует ожидаемому
2	Регистрация с коротким паролем	Открыт экран регистрации	1. Ввести имя, email. 2. Ввести пароль «123». 3. Нажать «Создать аккаунт»	Под полями отображено «Пароль должен быть не короче 6 символов»; сетевой запрос не выполняется	Соответствует ожидаемому
3	Вход с верными данными	Пользователь test@geo.ru зарегистрирован	1. На экране входа ввести email и пароль. 2. Нажать «Войти»	Открыт экран ленты; в журнале сервера запрос POST /api/auth/sync с кодом 200	Соответствует ожидаемому
4	Вход с неверным паролем	Пользователь зарегистрирован	1. Ввести email и пароль «wrong». 2. Нажать «Войти»	Сообщение «Неверный email или пароль»; экран входа сохранён	Соответствует ожидаемому
5	Просмотр ленты	Пользователь вошёл; в БД есть не менее трёх постов	1. Открыть вкладку «Лента». 2. Дождаться загрузки	Отображены посты в порядке убывания даты; у каждого — автор, место, текст, оценка, дата	Соответствует ожидаемому
6	Создание места	Пользователь вошёл; открыта вкладка «Карта»	1. Нажать «Добавить место». 2. Уточнить точку нажатием на карту. 3. Ввести название «Парк Горького», описание, выбрать «Парк». 4. Нажать «Сохранить место»	Возврат на карту; новый маркер в указанной точке; POST /api/places вернул 201; запись в таблице places	Соответствует ожидаемому
7	Создание места без названия	Открыт экран создания места	1. Оставить «Название» пустым. 2. Заполнить описание. 3. Нажать «Сохранить место»	Сообщение «Введите название места»; запрос не отправлен	Соответствует ожидаемому
8	Поиск мест по названию	В БД есть места «Парк Горького» и «Кофейня Точка»	1. Ввести «парк» в поисковую строку. 2. Нажать «Найти»	На карте остался маркер «Парк Горького»; GET /api/places?query=парк вернул одну запись	Соответствует ожидаемому
9	Поиск мест рядом	Эмулятор с заданными координатами 55.75, 37.61; разрешение на геолокацию выдано	1. Открыть вкладку «Карта». 2. Дождаться центрирования	Камера центрирована на позиции пользователя; загружены места в радиусе 10 км; поле distanceKm заполнено, список отсортирован по расстоянию	Соответствует ожидаемому
10	Публикация отзыва	Открыта карточка места	1. Ввести текст отзыва. 2. Выбрать оценку 5. 3. Нажать «Опубликовать»	Отзыв появился в начале списка; поле ввода очищено; POST /api/places/{id}/posts вернул 201	Соответствует ожидаемому
11	Отзыв с недопустимой оценкой (API)	Сервер в dev-режиме	1. curl -X POST /api/places/1/posts -H "Authorization: Bearer dev:test1" -d '{"text":"ok","rating":7}'	Ответ 400; тело {"error":"bad_request","message":"Оценка должна быть от 1 до 5"}	Соответствует ожидаемому

№	Название	Предусловие	Шаги	Ожидаемый результат	Фактический результат
12	Запрос без токена (API)	Сервер запущен	1. curl /api/feed без заголовка Authorization	Ответ 401 Unauthorized	Соответствует ожидаемому
13	Удаление чужого места (API)	Место с id = 1 создано пользователем test1	1. curl -X DELETE /api/places/1 -H "Authorization: Bearer dev:test2"	Ответ 403; сообщение «Удалять место может только его автор»; запись не удалена	Соответствует ожидаемому
14	Редактирование профиля	Открыта вкладка «Профиль»	1. Нажать «Редактировать». 2. Изменить имя на «Дени А.», описание. 3. Нажать «Сохранить»	Карточка показывает новые данные; PATCH /api/users/me вернул 200; данные сохранены после перезапуска	Соответствует ожидаемому
15	Отсутствие связи с сервером	Пользователь вошёл	1. Остановить сервер. 2. На вкладке «Лента» нажать «Обновить»	Сообщение «Нет связи с сервером. Проверьте, что сервер запущен.»; приложение продолжает работать	Соответствует ожидаемому
16	Выход из аккаунта	Пользователь вошёл	1. Открыть «Профиль». 2. Нажать кнопку выхода. 3. Перезапустить приложение	Открыт экран входа; после перезапуска сессия не восстановлена	Соответствует ожидаемому